

Geometry_MMALS_G1_SmoothResidual_SPD_v1_1_2

June 16, 2026

1 Geometry-MMALS G1 v1.1.2

1.1 Smooth Simplex-Residual Routing and Covariance-Aware Context Diagnostics

Status: complete C0 specification and executable Colab; five-seed pilot pending.

v1.1.1 successfully isolated the context-to-route bridge, but the prototype-dominated R3 router did not qualify: held-out order effects were not robust, local-continuity tails were large, causal specificity failed, and operational non-degradation failed. At the same time, route-swap ordering passed and R2/R3 reduced functional stress.

v1.1.2 therefore keeps the successful isolation controls and introduces two bounded repairs:

1. a **linear-first, smooth prototype residual** that cannot dominate the global route;
2. a **context-only continuity hinge** on the route probability simplex.

A separate diagnostic axis estimates regularized context covariances and compares log-Euclidean, affine-invariant, and Bures-Wasserstein SPD distances. The SPD axis is not a routing claim and does not assert that MMALS has learned a Riemannian manifold.

1.2 0. Release-integrity setup

The notebook installs package version 1.1.2. PDF verification remains an optional, independent layer.

```
[1]: from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

```
[2]: import os, sys, shutil, importlib, subprocess
from pathlib import Path

EXPECTED_PACKAGE_VERSION = "1.1.2"
EXPECTED_BUILD_REVISION = "smooth-residual-spd-diagnostics-c0-r1"
REPO_URL = "https://github.com/gharbonnier78/geometry-mmalls-g1.git"
REPO_DIR = Path("/content/geometry-mmalls-g1")
SRC_DIR = REPO_DIR / "src"
FORCE_REFRESH = False
```

```

if FORCE_REFRESH and REPO_DIR.exists():
    shutil.rmtree(REPO_DIR)
if not REPO_DIR.exists():
    subprocess.run(["git", "clone", REPO_URL, str(REPO_DIR)], check=True)

os.chdir(REPO_DIR)
subprocess.run(
    [sys.executable, "-m", "pip", "install", "--no-cache-dir", "--no-deps",
    ↪ "-e", str(REPO_DIR)],
    check=True,
)
if str(SRC_DIR) not in sys.path:
    sys.path.insert(0, str(SRC_DIR))
for name in list(sys.modules):
    if name == "geometry_mmalls" or name.startswith("geometry_mmalls."):
        del sys.modules[name]
importlib.invalidate_caches()
import geometry_mmalls

loaded_from = Path(geometry_mmalls.__file__).resolve()
assert loaded_from.parent == (SRC_DIR / "geometry_mmalls").resolve()
assert geometry_mmalls.__version__ == EXPECTED_PACKAGE_VERSION, (
    f"Expected {EXPECTED_PACKAGE_VERSION}, loaded {geometry_mmalls.__version__}.
    ↪ "
    "Push the v1.1.2 package and rerun with FORCE_REFRESH=True."
)
print("Geometry-MMALS", geometry_mmalls.__version__, "from", loaded_from)
print("Verification dependency policy: optional/lazy (pypdf not required for
    ↪ training)")

```

Geometry-MMALS 1.1.2 from /content/geometry-mmalls-g1/src/geometry_mmalls/__init__.py
 Verification dependency policy: optional/lazy (pypdf not required for training)

1.3 1. Configuration and preregistered treatments

R0-R3 are retained as historical controls. R4 tests the bounded residual architecture. R5 uses the same architecture plus the context-only continuity hinge. The common host bank and common functional cost remain frozen.

```

[3]: from pathlib import Path
import copy, hashlib, json, math, random, time, platform

import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F

```

```

import yaml
from scipy import stats
from scipy.stats import spearmanr
from sklearn.linear_model import Ridge
from sklearn.metrics import r2_score, roc_auc_score
from torch import nn
from torch.utils.data import DataLoader, Subset
from torchvision.datasets import MNIST

from geometry_mmalls.data import FixedAngleMNIST, MultiAngleMNIST
from geometry_mmalls.geometry import (
    paired_route_geometry_loss_stationary,
    paired_context_geometry_loss,
    context_path_spread_loss,
    cross_source_fiber_alignment_loss,
    factor_centroid_geometry_loss,
    normalized_stress,
)
from geometry_mmalls.metrics import (
    bootstrap_mean_ci,
    centroid_geometry_scores,
    grouped_geometry_scores,
    ridge_factor_probe,
)
from geometry_mmalls.model import GeometryMMALS, ResidualHost, SmallConvEncoder
from geometry_mmalls.functional_routing import (
    FunctionalRoutingMMALS,
    HybridDirectionalPrototypeRouter,
    SmoothSimplexResidualRouter,
    UniformRouter,
    LinearContextRouter,
    MLPContextRouter,
    PrototypeEnergyRouter,
    entropic_transport_cost,
    half_chord_distance,
    host_functional_diversity_loss,
    host_output_cost_matrix,
    host_specialization,
    host_territory,
    pairwise_functional_route_distances,
    parameter_count,
    reconstruct_route_from_root_probe,
    root_simplex_chord,
    territory_overlap,
    local_continuity_ratio,
    permute_hosts_and_routes,
    route_weighted_synthesis,

```

```

        usage_balance_loss,
        simplex_continuity_hinge_loss,
    )

from geometry_mmls.spd_geometry import (
    affine_invariant_distance,
    bures_wasserstein_distance,
    log_euclidean_distance,
    pairwise_spd_distances,
    regularized_covariance,
)

NOTEBOOK_VERSION = "1.1.2"
BUILD_REVISION = "smooth-residual-spd-diagnostics-c0-r1"
base_config = yaml.safe_load(Path("configs/rotated_mnist_g1.yaml").read_text())
protocol = yaml.safe_load(Path("configs/rotated_mnist_g1_v112.yaml").
    ↪read_text())
assert protocol["build_revision"] == BUILD_REVISION

# development: one seed; pilot: five seeds; qualification: ten seeds.
RUN_PROFILE = os.environ.get("G1_PROFILE", "pilot")
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
SPLIT_SEED = int(protocol["experiment"]["fixed_split_seed"])
SENSORY_SEED = int(protocol["experiment"]["fixed_sensory_seed"])

if RUN_PROFILE == "development":
    MODEL_SEEDS = [0]
    TRAIN_SOURCE_LIMIT, TEST_SOURCE_LIMIT = 256, 320
    SENSORY_SOURCE_LIMIT, SENSORY_TEST_LIMIT = 1500, 750
    SENSORY_EPOCHS, CONTEXT_STAGE_EPOCHS, BANK_STAGE_EPOCHS, ↵
    ↪ROUTER_STAGE_EPOCHS = 1, 1, 1, 1
    SOURCE_BATCH_SIZE, SOURCE_BOOTSTRAP_SAMPLES = 64, 300
    SENSORY_GATE = 0.80
elif RUN_PROFILE == "pilot":
    MODEL_SEEDS = list(map(int, protocol["experiment"]["pilot_model_seeds"]))
    TRAIN_SOURCE_LIMIT, TEST_SOURCE_LIMIT = 512, 640
    SENSORY_SOURCE_LIMIT, SENSORY_TEST_LIMIT = 2000, 1000
    SENSORY_EPOCHS, CONTEXT_STAGE_EPOCHS, BANK_STAGE_EPOCHS, ↵
    ↪ROUTER_STAGE_EPOCHS = 2, 2, 2, 2
    SOURCE_BATCH_SIZE = 64
    SOURCE_BOOTSTRAP_SAMPLES = ↵
    ↪int(protocol["statistics"]["source_bootstrap_samples"])
    SENSORY_GATE = 0.85
elif RUN_PROFILE == "qualification":
    MODEL_SEEDS = list(map(int, ↵
    ↪protocol["experiment"]["qualification_model_seeds"]))

```

```

    TRAIN_SOURCE_LIMIT = 0
    ↪int(base_config["paired_protocol"]["full_source_limit"])
    TEST_SOURCE_LIMIT, SENSORY_SOURCE_LIMIT, SENSORY_TEST_LIMIT = 2500, 12000, 0
    ↪5000
    SENSORY_EPOCHS = int(base_config["training"]["sensory_pretrain_epochs"])
    CONTEXT_STAGE_EPOCHS = BANK_STAGE_EPOCHS = ROUTER_STAGE_EPOCHS = 0
    ↪int(base_config["training"]["stage_epochs"])
    SOURCE_BATCH_SIZE = int(base_config["paired_protocol"]["source_batch_size"])
    SOURCE_BOOTSTRAP_SAMPLES = 0
    ↪int(protocol["statistics"]["source_bootstrap_samples"])
    SENSORY_GATE = 0.95
else:
    raise ValueError("G1_PROFILE must be development, pilot, or qualification")

TRAIN_ANGLES = list(map(float, base_config["data"]["train_angles"]))
CURRICULUM = list(map(float, protocol["experiment"]["curriculum"]))
DENSE_EVAL_ANGLES = list(map(float, 0
    ↪protocol["experiment"]["dense_eval_angles"]))
HELDOUT_ANGLES = [angle for angle in DENSE_EVAL_ANGLES if angle not in 0
    ↪TRAIN_ANGLES]
CONTEXT_DIM = 4
NUM_HOSTS = int(protocol["experiment"]["host_count"])
TAU = float(protocol["router_training"]["temperature"])
METHOD_SPECS = protocol["routers"]
METHODS = list(METHOD_SPECS)
PRIMARY_CONTRASTS = protocol["primary_contrasts"]

print({
    "version": NOTEBOOK_VERSION,
    "revision": BUILD_REVISION,
    "profile": RUN_PROFILE,
    "device": str(DEVICE),
    "seeds": MODEL_SEEDS,
    "methods": METHODS,
})

```

```

{'version': '1.1.2', 'revision': 'smooth-residual-spd-diagnostics-c0-r1',
'profile': 'pilot', 'device': 'cpu', 'seeds': [0, 1, 2, 3, 4], 'methods':
['r0_mlp', 'r1_linear', 'r2_prototype_energy', 'r3_hybrid_energy',
'r4_smooth_residual', 'r5_smooth_residual_continuity']}

```

1.4 2. Numerical and architectural self-tests

The tests cover R0-R5 simplex outputs, bounded residuals, finite gradients, the continuity hinge, common-cost Sinkhorn transport, permutation invariance, and SPD matrix distances.

```

[4]: torch.manual_seed(112)
probe_context = F.normalize(torch.randn(16, CONTEXT_DIM), dim=-1)
routers = [
    MLPContextRouter(CONTEXT_DIM, NUM_HOSTS),
    LinearContextRouter(CONTEXT_DIM, NUM_HOSTS),
    PrototypeEnergyRouter(CONTEXT_DIM, NUM_HOSTS),
    HybridDirectionalPrototypeRouter(CONTEXT_DIM, NUM_HOSTS),
    SmoothSimplexResidualRouter(CONTEXT_DIM, NUM_HOSTS),
]
for router in routers:
    route = router(probe_context)
    assert route.shape == (16, NUM_HOSTS)
    assert torch.isfinite(route).all()
    assert torch.allclose(route.sum(-1), torch.ones(16), atol=1e-6)

smooth = routers[-1]
smooth.initialize_from_contexts(probe_context, seed=0)
base_logits, residual, gate = smooth.components(probe_context)
assert residual.abs().max() <= smooth.residual_cap + 1e-6
assert torch.all((gate >= 0) & (gate <= 1))
loss = smooth(probe_context).square().mean()
loss.backward()
assert all(p.grad is None or torch.isfinite(p.grad).all() for p in smooth.
    ↪parameters())

route_tensor = torch.stack([router(probe_context[:8]) for router in routers[:
    ↪2]], dim=1)
context_tensor = probe_context[:8, None, :].expand(-1, 2, -1).clone()
context_tensor[:, 1] = F.normalize(context_tensor[:, 1] + 0.05 * torch.
    ↪randn_like(context_tensor[:, 1]), dim=-1)
continuity_loss = simplex_continuity_hinge_loss(route_tensor, context_tensor,
    ↪kappa=1.0)
assert torch.isfinite(continuity_loss)

host_probe = torch.randn(32, NUM_HOSTS, 24)
cost_probe = host_output_cost_matrix(host_probe)
route_a = torch.softmax(torch.randn(32, NUM_HOSTS), dim=-1)
route_b = torch.softmax(torch.randn(32, NUM_HOSTS), dim=-1)
assert torch.isfinite(entropic_transport_cost(route_a, route_b, cost_probe)).
    ↪all()

permutation = torch.tensor([2, 0, 3, 1])
route_p, host_p = permute_hosts_and_routes(route_a, host_probe, permutation)
assert torch.allclose(
    route_weighted_synthesis(route_a, host_probe),
    route_weighted_synthesis(route_p, host_p),
    atol=1e-6,

```

```

)

samples_a = torch.randn(64, CONTEXT_DIM)
samples_b = torch.randn(64, CONTEXT_DIM) + 0.2
cov_a = regularized_covariance(samples_a)
cov_b = regularized_covariance(samples_b)
for value in [
    log_euclidean_distance(cov_a, cov_b),
    affine_invariant_distance(cov_a, cov_b),
    bures_wasserstein_distance(cov_a, cov_b),
]:
    assert torch.isfinite(value) and value >= 0
print("R0-R5, residual bound, continuity, Sinkhorn, permutation, and SPD gates:
↪PASS")

```

R0-R5, residual bound, continuity, Sinkhorn, permutation, and SPD gates: PASS

1.5 3. Source-disjoint partitions and frozen sensory grove

Six partitions isolate common-cost construction, probe fitting, probe evaluation, tangent fitting, causal evaluation, and route-swap evaluation.

```

[5]: root = Path(base_config["data"]["root"])
base_train = MNIST(root=str(root), train=True, download=True)
base_test = MNIST(root=str(root), train=False, download=True)

split_rng = np.random.default_rng(SPLIT_SEED)
train_perm = split_rng.permutation(len(base_train))
test_perm = split_rng.permutation(len(base_test))
sensory_indices = train_perm[:SENSORY_SOURCE_LIMIT].tolist()
train_indices = train_perm[TRAIN_SOURCE_LIMIT:].tolist()
test_indices = test_perm[:TEST_SOURCE_LIMIT].tolist()
sensory_test_indices = test_perm[:SENSORY_TEST_LIMIT].tolist()

partition_arrays = np.array_split(np.asarray(test_indices), 6)
metric_indices = partition_arrays[0].tolist()
probe_fit_indices = partition_arrays[1].tolist()
probe_eval_indices = partition_arrays[2].tolist()
tangent_fit_indices = partition_arrays[3].tolist()
causal_eval_indices = partition_arrays[4].tolist()
swap_eval_indices = partition_arrays[5].tolist()

def checksum(values):
    return hashlib.sha256(",".join(map(str, values)).encode()).hexdigest()

def generator(seed):

```

```

g = torch.Generator(); g.manual_seed(int(seed)); return g

def fixed_loader(angle, train, indices, shuffle, loader_seed=0, batch_size=128):
    ds = Subset(FixedAngleMNIST(root, angle=angle, train=train, download=True),
        indices)
    return DataLoader(
        ds, batch_size=batch_size, shuffle=shuffle, num_workers=0,
        generator=generator(loader_seed) if shuffle else None,
    )

def multi_loader(angles, train, indices, shuffle, loader_seed=0,
    batch_size=None):
    ds = MultiAngleMNIST(root, angles=angles, train=train, indices=indices,
        download=True)
    return DataLoader(
        ds, batch_size=batch_size or SOURCE_BATCH_SIZE, shuffle=shuffle,
        num_workers=0,
        generator=generator(loader_seed) if shuffle else None,
    )

split_manifest = {
    "split_seed": SPLIT_SEED,
    "train_sha256": checksum(train_indices),
    "test_sha256": checksum(test_indices),
    "metric_sha256": checksum(metric_indices),
    "probe_fit_sha256": checksum(probe_fit_indices),
    "probe_eval_sha256": checksum(probe_eval_indices),
    "tangent_fit_sha256": checksum(tangent_fit_indices),
    "causal_eval_sha256": checksum(causal_eval_indices),
    "swap_eval_sha256": checksum(swap_eval_indices),
}

random.seed(SENSORY_SEED); np.random.seed(SENSORY_SEED); torch.
    manual_seed(SENSORY_SEED)
if torch.cuda.is_available(): torch.cuda.manual_seed_all(SENSORY_SEED)
latent_dim = int(base_config["model"]["latent_dim"])
encoder = SmallConvEncoder(latent_dim).to(DEVICE)
sensory_head = nn.Linear(latent_dim, 10).to(DEVICE)
sensory_optimizer = torch.optim.AdamW(
    list(encoder.parameters()) + list(sensory_head.parameters()),
    lr=float(base_config["training"]["learning_rate"]),
    weight_decay=float(base_config["training"]["weight_decay"]),
)
sensory_history = []
for epoch in range(SENSORY_EPOCHS):

```



```

total = correct = 0; loss_sum = 0.0
encoder.train(); sensory_head.train()
for x, y, _, _ in fixed_loader(0.0, True, sensory_indices, True,
↳SENSORY_SEED * 1000 + epoch):
    x, y = x.to(DEVICE), y.to(DEVICE)
    logits = sensory_head(encoder(x)); loss = F.cross_entropy(logits, y)
    sensory_optimizer.zero_grad(set_to_none=True); loss.backward()
    torch.nn.utils.clip_grad_norm_(list(encoder.parameters()) +
↳list(sensory_head.parameters()), 5.0)
    sensory_optimizer.step()
    total += y.numel(); correct += (logits.argmax(1) == y).sum().item()
    loss_sum += float(loss.detach()) * y.numel()
    sensory_history.append({"epoch": epoch, "loss": loss_sum / total,
↳"train_accuracy": correct / total})
    print(sensory_history[-1])

@torch.no_grad()
def evaluate_sensory():
    encoder.eval(); sensory_head.eval(); total = correct = 0
    for x, y, _, _ in fixed_loader(0.0, False, sensory_test_indices, False,
↳batch_size=256):
        x, y = x.to(DEVICE), y.to(DEVICE)
        correct += (sensory_head(encoder(x)).argmax(1) == y).sum().item();
↳total += y.numel()
    return correct / total

sensory_accuracy = evaluate_sensory()
assert RUN_PROFILE != "qualification" or sensory_accuracy >= SENSORY_GATE
sensory_state = copy.deepcopy(encoder.state_dict())
print("Sensory gate:", sensory_accuracy, "threshold:", SENSORY_GATE)

```

```

{'epoch': 0, 'loss': 1.5406257815361022, 'train_accuracy': 0.606}
{'epoch': 1, 'loss': 0.5623613724708557, 'train_accuracy': 0.909}
Sensory gate: 0.942 threshold: 0.85

```

1.6 4. Reproduce and freeze the v1.0.9 aligned context checkpoint

The aligned context stage is unchanged from v1.1.0. Only the sensory and context encoders are transferred to the bridge-isolation phase.

```

[6]: def state_hash(state_dict):
    h = hashlib.sha256()
    for name in sorted(state_dict):
        tensor = state_dict[name].detach().cpu().contiguous()
        h.update(name.encode()); h.update(tensor.numpy().tobytes())
    return h.hexdigest()

```

```

def epoch_seed(model_seed, stage, epoch, offset=0):
    return int(model_seed) * 1_000_000 + stage * 10_000 + epoch + int(offset)

def build_context_source_model():
    enc = SmallConvEncoder(latent_dim).to(DEVICE); enc.
    ↪load_state_dict(sensory_state)
    return GeometryMMALS(
        enc, latent_dim=latent_dim, context_dim=CONTEXT_DIM,
    ↪num_hosts=NUM_HOSTS,
        host_hidden_dim=int(base_config["model"]["host_hidden_dim"]),
        freeze_encoder=True, bottleneck_hidden_dim=64,
    ).to(DEVICE)

def context_source_forward(model, flat):
    z0 = model.encode(flat)
    raw, context = model.infer_context(z0)
    route = torch.softmax(model.context_bottleneck_router(context) / TAU,
    ↪dim=-1)
    return model.synthesize(z0, context, route, context_raw=raw)

def train_aligned_context_checkpoint(model_seed):
    torch.manual_seed(model_seed + 1090)
    model = build_context_source_model()
    params = [p for p in model.parameters() if p.requires_grad]
    optimizer = torch.optim.AdamW(
        params,
        lr=float(base_config["training"]["learning_rate"]),
        weight_decay=float(base_config["training"]["weight_decay"]),
    )
    rows = []
    for stage, current_angle in enumerate(CURRICULUM):
        seen = CURRICULUM[:stage + 1]
        totals = {key: 0.0 for key in [
            "loss", "current_ce", "anchor_ce", "route_geo", "context_geo",
            "context_spread", "fiber", "centroid", "host_diversity",
        ]}
        totals.update({"forward_images": 0, "optimizer_steps": 0,
    ↪"source_examples": 0})
        model.train(); start = time.perf_counter()
        for epoch in range(CONTEXT_STAGE_EPOCHS):
            loader = multi_loader(seen, True, train_indices, True,
    ↪epoch_seed(model_seed, stage, epoch, 109))
            for views, y, factors, _ in loader:

```

```

        batch, angle_count = views.shape[:2]
        flat = views.reshape(-1, *views.shape[2:]).to(DEVICE)
        y, factors = y.to(DEVICE), factors.to(DEVICE)
        trace = context_source_forward(model, flat)
        logits = trace.logits.reshape(batch, angle_count, -1)
        routes = trace.route.reshape(batch, angle_count, -1)
        contexts = trace.context.reshape(batch, angle_count, -1)
        hosts = trace.host_outputs.reshape(batch, angle_count,
↪NUM_HOSTS, -1)
        current_ce = F.cross_entropy(logits[:, -1], y)
        anchor_ce = (
            F.cross_entropy(
                logits[:, :-1].reshape(-1, logits.shape[-1]),
                y[:, None].expand(-1, angle_count - 1).reshape(-1),
            ) if angle_count > 1 else logits.sum() * 0.0
        )
        route_geo = paired_route_geometry_loss_stationary(routes,
↪factors)

        context_geo = paired_context_geometry_loss(contexts, factors)
        spread = context_path_spread_loss(contexts, 0.05)
        fiber = cross_source_fiber_alignment_loss(contexts, factors)
        centroid = factor_centroid_geometry_loss(contexts, factors)
        diversity = host_functional_diversity_loss(hosts[:, -1])
        loss = (
            current_ce + 0.10 * anchor_ce + 0.10 * route_geo
            + 0.10 * context_geo + 0.05 * spread + 0.05 * fiber
            + 0.05 * centroid + 0.05 * diversity
        )
        components = torch.stack([loss, current_ce, anchor_ce,
↪route_geo, context_geo, spread, fiber, centroid, diversity])
        if not torch.isfinite(components).all():
            raise FloatingPointError(f"Non-finite context-source loss,
↪seed={model_seed}, stage={stage}")
        optimizer.zero_grad(set_to_none=True); loss.backward()
        grad = torch.nn.utils.clip_grad_norm_(params, 5.0)
        if not torch.isfinite(torch.as_tensor(grad)):
            raise FloatingPointError("Non-finite context-source
↪gradient")
        optimizer.step()
        totals["source_examples"] += batch; totals["forward_images"] +=
↪batch * angle_count
        totals["optimizer_steps"] += 1
        for key, value in {
            "loss": loss, "current_ce": current_ce, "anchor_ce":
↪anchor_ce,
            "route_geo": route_geo, "context_geo": context_geo,

```

```

        "context_spread": spread, "fiber": fiber, "centroid":  

↪centroid,  

        "host_diversity": diversity,  

    }.items():  

        totals[key] += float(value.detach()) * batch  

    count = max(totals["source_examples"], 1)  

    row = {  

        "model_seed": model_seed, "stage": stage, "current_angle":  

↪current_angle,  

        "seen_angles": json.dumps(seen), "wall_seconds": time.  

↪perf_counter() - start,  

        "forward_images": totals["forward_images"], "optimizer_steps":  

↪totals["optimizer_steps"],  

        **{key: totals[key] / count for key in totals if key not in  

↪{"forward_images", "optimizer_steps", "source_examples"}},  

    }  

    rows.append(row); print("context", row)  

    checkpoint = {  

        "encoder": copy.deepcopy(model.encoder.state_dict()),  

        "context_net": copy.deepcopy(model.context_net.state_dict()),  

    }  

    return model, checkpoint, state_hash(checkpoint["context_net"]), pd.  

↪DataFrame(rows)  

  

@torch.no_grad()  

def collect_context_bank(source_model, angles, indices):  

    contexts = []  

    source_model.eval()  

    for views, _, _, _ in multi_loader(angles, True, indices, False):  

        flat = views.reshape(-1, *views.shape[2:]).to(DEVICE)  

        z0 = source_model.encode(flat)  

        _, context = source_model.infer_context(z0)  

        contexts.append(context.cpu())  

    return torch.cat(contexts, dim=0)

```

1.7 5. Construct and freeze one common host bank

A parameter-free uniform router trains a neutral bank once per seed. The bank uses classification, retention anchoring, and host-output diversity, but no factor label in routing. It is then frozen.

The shared functional cost is

$$C_{hk}^* = \mathbb{E}_x \frac{\|g_h(z_0(x)) - g_k(z_0(x))\|_2^2}{d_M}.$$

```

[7]: def build_common_bank_model(checkpoint, model_seed):
    torch.manual_seed(model_seed + 1110)
    enc = SmallConvEncoder(latent_dim); enc.
    ↪load_state_dict(checkpoint["encoder"])
    context_net = nn.Sequential(nn.Linear(latent_dim, 64), nn.GELU(), nn.
    ↪Linear(64, CONTEXT_DIM))
    context_net.load_state_dict(checkpoint["context_net"])
    hosts = [ResidualHost(latent_dim,
    ↪int(base_config["model"]["host_hidden_dim"])) for _ in range(NUM_HOSTS)]
    return FunctionalRoutingMMALS(
        enc, context_net, UniformRouter(NUM_HOSTS), hosts,
        nn.LayerNorm(latent_dim), nn.Linear(latent_dim, 10),
        freeze_encoder=True, freeze_context=True,
    ).to(DEVICE)

def train_common_host_bank(model_seed, checkpoint):
    model = build_common_bank_model(checkpoint, model_seed)
    params = [p for name, p in model.named_parameters() if p.requires_grad and
    ↪not name.startswith("router.")]
    optimizer = torch.optim.AdamW(params,
    ↪lr=float(base_config["training"]["learning_rate"]),
    ↪weight_decay=float(base_config["training"]["weight_decay"]))
    weights = protocol["common_host_bank"]["loss"]
    rows = []
    for stage, current_angle in enumerate(CURRICULUM):
        seen = CURRICULUM[:stage + 1]
        totals = {k: 0.0 for k in ["loss", "current_ce", "anchor_ce",
    ↪"host_diversity"]}
        totals.update({"source_examples": 0, "forward_images": 0,
    ↪"optimizer_steps": 0})
        start = time.perf_counter(); model.train()
        for epoch in range(BANK_STAGE_EPOCHS):
            for views, y, _, _ in multi_loader(seen, True, train_indices, True,
    ↪epoch_seed(model_seed, stage, epoch, 511)):
                batch, angle_count = views.shape[:2]
                y = y.to(DEVICE)
                trace = model(views.reshape(-1, *views.shape[2:])).to(DEVICE),
    ↪temperature=TAU)
                logits = trace.logits.reshape(batch, angle_count, -1)
                hosts = trace.host_outputs.reshape(batch, angle_count,
    ↪NUM_HOSTS, -1)
                current_ce = F.cross_entropy(logits[:, -1], y)
                anchor_ce = F.cross_entropy(logits[:, :-1].reshape(-1, logits.
    ↪shape[-1]), y[:, None].expand(-1, angle_count - 1).reshape(-1)) if
    ↪angle_count > 1 else logits.sum() * 0.0
                diversity = host_functional_diversity_loss(hosts[:, -1])

```

```

        loss = float(weights["current_ce"]) * current_ce +
↪float(weights["retention_anchor"]) * anchor_ce +
↪float(weights["host_diversity"]) * diversity
        optimizer.zero_grad(set_to_none=True); loss.backward()
        grad = torch.nn.utils.clip_grad_norm_(params, 5.0)
        if not torch.isfinite(torch.as_tensor(grad)): raise
↪FloatingPointError("Non-finite common-bank gradient")
        optimizer.step()
        totals["source_examples"] += batch; totals["forward_images"] +=
↪batch * angle_count; totals["optimizer_steps"] += 1
        for key, value in {"loss": loss, "current_ce": current_ce,
↪"anchor_ce": anchor_ce, "host_diversity": diversity}.items(): totals[key] +=
↪float(value.detach()) * batch
        count = max(totals["source_examples"], 1)
        rows.append({"model_seed": model_seed, "stage": stage, "current_angle":
↪current_angle, "seen_angles": json.dumps(seen), "wall_seconds": time.
↪perf_counter() - start, "forward_images": totals["forward_images"],
↪"optimizer_steps": totals["optimizer_steps"], **{k: totals[k]/count for k in
↪["loss", "current_ce", "anchor_ce", "host_diversity"]}})
        state = {"hosts": copy.deepcopy(model.hosts.state_dict()), "synthesis_norm":
↪copy.deepcopy(model.synthesis_norm.state_dict()), "classifier": copy.
↪deepcopy(model.classifier.state_dict())}
        flat = {**{f"hosts.{k}":v for k,v in state["hosts"].items()},
↪**{f"synthesis_norm.{k}":v for k,v in state["synthesis_norm"].items()},
↪**{f"classifier.{k}":v for k,v in state["classifier"].items()}}
        return model, state, state_hash(flat), pd.DataFrame(rows)

@torch.no_grad()
def build_common_host_cost(bank_model):
    bank_model.eval(); outputs = []
    for views, _, _ in multi_loader(DENSE_EVAL_ANGLES, False,
↪metric_indices, False):
        trace = bank_model(views.reshape(-1, *views.shape[2:]).to(DEVICE),
↪temperature=TAU)
        outputs.append(trace.host_outputs.cpu())
    return host_output_cost_matrix(torch.cat(outputs), normalize=True)

```

1.8 6. Build and train R0-R5 on the frozen bank

Only router parameters are trainable. R4 is the smooth residual architecture; R5 is the same architecture with a non-factor-supervised continuity hinge.

```

[8]: def build_router(method, context_bank, model_seed):
    spec = METHOD_SPECS[method]
    kind = spec["type"]
    torch.manual_seed(model_seed + int(spec["seed_offset"]))

```

```

    if kind == "mlp_context_router":
        router, diag = MLPContextRouter(CONTEXT_DIM, NUM_HOSTS,
        ↪hidden_dim=int(spec["hidden_dim"])), {}
    elif kind == "linear_context_router":
        router, diag = LinearContextRouter(CONTEXT_DIM, NUM_HOSTS), {}
    elif kind == "prototype_energy_router":
        router = PrototypeEnergyRouter(CONTEXT_DIM, NUM_HOSTS,
        ↪sigma_min=float(spec["sigma_min"]), temperature=TAU)
        diag = router.initialize_from_contexts(context_bank.to(DEVICE),
        ↪seed=model_seed + int(spec["seed_offset"]),
        ↪iterations=int(spec["spherical_kmeans_iterations"]))
    elif kind == "hybrid_directional_prototype_router":
        router = HybridDirectionalPrototypeRouter(
            CONTEXT_DIM, NUM_HOSTS,
            sigma_min=float(spec["sigma_min"]), temperature=TAU,
            prototype_weight=float(spec["prototype_weight"]),
            directional_weight=float(spec["directional_weight"]),
            learnable_mixture=bool(spec["learnable_mixture"]),
        )
        diag = router.initialize_from_contexts(context_bank.to(DEVICE),
        ↪seed=model_seed + int(spec["seed_offset"]),
        ↪iterations=int(spec["spherical_kmeans_iterations"]))
    elif kind == "smooth_simplex_residual_router":
        router = SmoothSimplexResidualRouter(
            CONTEXT_DIM, NUM_HOSTS,
            sigma_min=float(spec["sigma_min"]), temperature=TAU,
            residual_cap=float(spec["residual_cap"]),
            gate_hidden_dim=int(spec["gate_hidden_dim"]),
            learnable_cap=bool(spec["learnable_cap"]),
        )
        diag = router.initialize_from_contexts(context_bank.to(DEVICE),
        ↪seed=model_seed + int(spec["seed_offset"]),
        ↪iterations=int(spec["spherical_kmeans_iterations"]))
    else:
        raise ValueError(kind)
    return router.to(DEVICE), diag

def build_routing_model(method, checkpoint, bank_state, context_bank,
    ↪model_seed):
    enc = SmallConvEncoder(latent_dim)
    enc.load_state_dict(checkpoint["encoder"])
    context_net = nn.Sequential(nn.Linear(latent_dim, 64), nn.GELU(), nn.
    ↪Linear(64, CONTEXT_DIM))
    context_net.load_state_dict(checkpoint["context_net"])

```

```

    hosts = [ResidualHost(latent_dim,
↪int(base_config["model"]["host_hidden_dim"])) for _ in range(NUM_HOSTS)]
    host_list = nn.ModuleList(hosts)
    host_list.load_state_dict(bank_state["hosts"])
    synthesis_norm = nn.LayerNorm(latent_dim)
    synthesis_norm.load_state_dict(bank_state["synthesis_norm"])
    classifier = nn.Linear(latent_dim, 10)
    classifier.load_state_dict(bank_state["classifier"])
    router, diag = build_router(method, context_bank, model_seed)
    model = FunctionalRoutingMMALS(enc, context_net, router, hosts,
↪synthesis_norm, classifier, freeze_encoder=True, freeze_context=True).
↪to(DEVICE)

    for module in [model.hosts, model.synthesis_norm, model.classifier]:
        for parameter in module.parameters():
            parameter.requires_grad_(False)
    return model, diag

@torch.no_grad()
def evaluate_routing_model(model, method, model_seed, stage, angles):
    rows = []
    model.eval()
    for angle in angles:
        total = correct = 0
        nll_sum = 0.0
        for x, y, _, _ in fixed_loader(angle, False, test_indices, False,
↪batch_size=256):
            x, y = x.to(DEVICE), y.to(DEVICE)
            trace = model(x, temperature=TAU)
            total += y.numel()
            correct += int((trace.logits.argmax(1) == y).sum())
            nll_sum += float(F.cross_entropy(trace.logits, y, reduction="sum"))
        rows.append({
            "model_seed": model_seed, "method": method, "stage": stage,
            "angle": float(angle), "angle_type": "train" if angle in
↪TRAIN_ANGLES else "heldout",
            "accuracy": correct / total, "nll": nll_sum / total,
        })
    return rows

def train_router_treatment(model_seed, method, model, checkpoint_hash,
↪bank_hash):
    params = [p for p in model.router.parameters() if p.requires_grad]
    optimizer = torch.optim.AdamW(
        params,
        lr=float(base_config["training"]["learning_rate"]),

```



```

        weight_decay=float(base_config["training"]["weight_decay"]),
    )
    weights = protocol["router_training"]["loss"]
    continuity_cfg = protocol["router_training"]["continuity"]
    continuity_weight = float(METHOD_SPECS[method].get("continuity_weight", 0.
↪0))
    stage_rows, evaluation_rows = [], []
    for stage, current_angle in enumerate(CURRICULUM):
        seen = CURRICULUM[:stage + 1]
        totals = {key: 0.0 for key in ["loss", "current_ce", "anchor_ce", ↪
↪"balance", "continuity"]}
        totals.update({"source_examples": 0, "forward_images": 0, ↪
↪"optimizer_steps": 0})
        start = time.perf_counter()
        model.train()
        for epoch in range(ROUTER_STAGE_EPOCHS):
            loader = multi_loader(seen, True, train_indices, True, ↪
↪epoch_seed(model_seed, stage, epoch, 711))
            for views, y, _, _ in loader:
                batch, angle_count = views.shape[:2]
                y = y.to(DEVICE)
                trace = model(views.reshape(-1, *views.shape[2:]).to(DEVICE), ↪
↪temperature=TAU)
                logits = trace.logits.reshape(batch, angle_count, -1)
                routes = trace.route.reshape(batch, angle_count, -1)
                contexts = trace.context.reshape(batch, angle_count, -1)
                current_ce = F.cross_entropy(logits[:, -1], y)
                anchor_ce = F.cross_entropy(
                    logits[:, :-1].reshape(-1, logits.shape[-1]),
                    y[:, None].expand(-1, angle_count - 1).reshape(-1),
                ) if angle_count > 1 else logits.sum() * 0.0
                balance = usage_balance_loss(routes)
                continuity = simplex_continuity_hinge_loss(
                    routes, contexts,
                    kappa=float(continuity_cfg["kappa"]),
                    margin=float(continuity_cfg["margin"]),
                )
            loss = (
                float(weights["current_ce"]) * current_ce
                + float(weights["retention_anchor"]) * anchor_ce
                + float(weights["usage_balance"]) * balance
                + continuity_weight * continuity
            )
            optimizer.zero_grad(set_to_none=True)
            loss.backward()
            grad = torch.nn.utils.clip_grad_norm_(params, 5.0)
            if not torch.isfinite(torch.as_tensor(grad)):

```

```

        raise FloatingPointError("Non-finite router gradient")
    optimizer.step()
    if hasattr(model.router, "project_parameters_"):
        model.router.project_parameters_()
    elif isinstance(model.router, PrototypeEnergyRouter):
        model.router.project_prototypes_()
    totals["source_examples"] += batch
    totals["forward_images"] += batch * angle_count
    totals["optimizer_steps"] += 1
    for key, value in {
        "loss": loss, "current_ce": current_ce, "anchor_ce":
↪anchor_ce,
        "balance": balance, "continuity": continuity,
    }.items():
        totals[key] += float(value.detach()) * batch
    count = max(totals["source_examples"], 1)
    stage_rows.append({
        "model_seed": model_seed, "method": method, "stage": stage,
        "current_angle": current_angle, "seen_angles": json.dumps(seen),
        "context_checkpoint_hash": checkpoint_hash, "common_bank_hash":
↪bank_hash,
        "router_parameters": parameter_count(model.router),
        "continuity_weight": continuity_weight,
        "forward_images": totals["forward_images"],
        "optimizer_steps": totals["optimizer_steps"],
        "wall_seconds": time.perf_counter() - start,
        **{key: totals[key] / count for key in ["loss", "current_ce",
↪"anchor_ce", "balance", "continuity"]},
    })
    evaluation_rows.extend(evaluate_routing_model(model, method,
↪model_seed, stage, DENSE_EVAL_ANGLES))
    return model, pd.DataFrame(stage_rows), pd.DataFrame(evaluation_rows)

```

1.9 7. Traces and preservation audit

Context vectors and host outputs must be numerically identical across all router treatments for the same source and angle.

```

[9]: @torch.no_grad()
def collect_trace(model, method, model_seed, angles, indices,
↪include_hosts=False):
    rows = []
    model.eval()
    for views, labels, factors, source_ids in multi_loader(angles, False,
↪indices, False):
        batch, angle_count = views.shape[:2]

```

```

        trace = model(views.reshape(-1, *views.shape[2:]).to(DEVICE),
↪temperature=TAU)
        logits = trace.logits.reshape(batch, angle_count, -1)
        arrays = {
            "context": trace.context.reshape(batch, angle_count, -1).cpu().
↪numpy(),
            "route": trace.route.reshape(batch, angle_count, -1).cpu().numpy(),
            "z_mm": trace.z_mm.reshape(batch, angle_count, -1).cpu().numpy(),
            "logits": logits.cpu().numpy(),
        }
        if include_hosts:
            arrays["host_outputs"] = trace.host_outputs.reshape(batch,
↪angle_count, NUM_HOSTS, -1).cpu().numpy()
            log_probs = F.log_softmax(logits, dim=-1).cpu().numpy()
            for source in range(batch):
                label = int(labels[source])
                for angle_index in range(angle_count):
                    row = {
                        "model_seed": model_seed,
                        "method": method,
                        "source_index": int(source_ids[source]),
                        "label": label,
                        "angle": float(factors[source, angle_index]),
                        "correct": int(arrays["logits"][source, angle_index].
↪argmax() == label),
                        "true_log_prob": float(log_probs[source, angle_index,
↪label]),
                        "context": arrays["context"][source, angle_index],
                        "route": arrays["route"][source, angle_index],
                        "z_mm": arrays["z_mm"][source, angle_index],
                    }
                    if include_hosts:
                        row["host_outputs"] = arrays["host_outputs"][source,
↪angle_index]
                    rows.append(row)
            return pd.DataFrame(rows)

def stack(frame, column):
    return np.stack(frame[column].to_numpy())

def preservation_table(trace):
    reference = trace[trace.method == METHODS[0]][["source_index", "angle",
↪"context", "host_outputs"]].copy()
    rows = []

```

```

    for method in METHODS[1:]:
        other = trace[trace.method == method][["source_index", "angle",
↪ "context", "host_outputs"]]
        merged = reference.merge(other, on=["source_index", "angle"],
↪ suffixes=("_ref", "_other"))
        dc = np.stack(merged.context_ref) - np.stack(merged.context_other)
        dh = np.stack(merged.host_outputs_ref) - np.stack(merged.
↪ host_outputs_other)
        rows.append({
            "method": method,
            "reference": METHODS[0],
            "max_abs_context_delta": float(np.abs(dc).max()),
            "max_abs_host_output_delta": float(np.abs(dh).max()),
            "mean_context_l2_delta": float(np.linalg.norm(dc, axis=1).mean()),
            "mean_host_l2_delta": float(np.linalg.norm(dh.reshape(len(dh), -1),
↪ axis=1).mean()),
        })
    return pd.DataFrame(rows)

```

1.10 8. Covariance/SPD context diagnostics

For each dense angle, context vectors across source identities define a regularized covariance matrix. The notebook compares centroid-only geometry with log-Euclidean, affine-invariant, Bures-Wasserstein, and combined centroid-plus-covariance metrics. This axis is diagnostic only.

```

[10]: def _normalize_distance_matrix(matrix):
    matrix = np.asarray(matrix, dtype=float)
    nonzero = matrix[matrix > 1e-12]
    scale = np.median(nonzero) if nonzero.size else 1.0
    return matrix / max(scale, 1e-12)

def spd_context_diagnostics(frame, model_seed):
    cfg = protocol["spd_context_diagnostics"]
    angles = np.asarray(sorted(frame.angle.unique()), dtype=float)
    means, covariances, covariance_rows = [], [], []
    for angle in angles:
        values = torch.tensor(stack(frame[frame.angle == angle], "context"),
↪ dtype=torch.float64)
        mean = F.normalize(values.mean(0), dim=-1)
        covariance = regularized_covariance(
            values,
            shrinkage=float(cfg["shrinkage"]),
            eps=float(cfg["epsilon"]),
        )
        means.append(mean)
        covariances.append(covariance)

```

```

        for i in range(covariance.shape[0]):
            for j in range(covariance.shape[1]):
                covariance_rows.append({
                    "model_seed": model_seed, "angle": float(angle),
                    "row": i, "column": j, "value": float(covariance[i, j]),
                })
    means = torch.stack(means)
    covariances = torch.stack(covariances)
    centroid = np.zeros((len(angles), len(angles)), dtype=float)
    for i in range(len(angles)):
        for j in range(i + 1, len(angles)):
            value = float(half_chord_distance(means[i:i+1], means[j:j+1]))
            centroid[i, j] = centroid[j, i] = value
    matrices = {"centroid": centroid}
    for metric in cfg["metrics"]:
        matrices[metric] = pairwise_spd_distances(covariances, metric=metric).
        ↪cpu().numpy()
    matrices["centroid_plus_log_euclidean"] = (
        float(cfg["combined_metric"]["centroid_weight"]) *
        ↪_normalize_distance_matrix(centroid)
        + float(cfg["combined_metric"]["covariance_weight"]) *
        ↪_normalize_distance_matrix(matrices["log_euclidean"])
    )
    reference = pairwise_factor_distance(angles)
    mask = np.triu(np.ones_like(reference, dtype=bool), k=1)
    pair_rows, summary_rows = [], []
    far_threshold = float(cfg["far_pair_threshold_degrees"])
    near_threshold = float(cfg["near_pair_threshold_degrees"])
    for metric, matrix in matrices.items():
        rho, stress = geometry_score_from_matrices(reference, matrix)
        y_true, y_score = [], []
        for i in range(len(angles)):
            for j in range(i + 1, len(angles)):
                gap = abs(float(angles[i] - angles[j]))
                category = "near" if gap <= near_threshold else "far" if gap >=
                ↪far_threshold else "mid"
                pair_rows.append({
                    "model_seed": model_seed, "metric": metric,
                    "angle_a": float(angles[i]), "angle_b": float(angles[j]),
                    "factor_gap": gap, "category": category, "distance":
                ↪float(matrix[i, j]),
                })
                if category != "mid":
                    y_true.append(1 if category == "far" else 0)
                    y_score.append(float(matrix[i, j]))
        auc = float(roc_auc_score(y_true, y_score)) if len(set(y_true)) == 2
        ↪else float("nan")

```

```

summary_rows.append({
    "model_seed": model_seed, "metric": metric,
    "rho": rho, "stress": stress, "near_far_auc": auc,
})
return pd.DataFrame(covariance_rows), pd.DataFrame(pair_rows), pd.
↪DataFrame(summary_rows)

```

1.11 9. Common functional geometry and local continuity distributions

The primary functional distance is now evaluated with the same C^* for all treatments. Local stability is measured through the complete distribution of

$$L_{local} = \frac{d_H(R(c_a), R(c_b))}{d_C(c_a, c_b) + \epsilon},$$

including its 90th, 95th, 99th percentiles and maximum.

```

[11]: @torch.no_grad()
def build_host_cost(model, angles, indices):
    model.eval(); running = None; count = 0
    for views, _, _ in multi_loader(angles, False, indices, False):
        trace = model(views.reshape(-1, *views.shape[2:]).to(DEVICE),
↪temperature=TAU)
        outputs = trace.host_outputs
        delta = outputs[:, :, None, :] - outputs[:, None, :, :]
        batch_cost = torch.square(delta).sum(-1).sum(0) / float(outputs.
↪shape[-1])
        running = batch_cost if running is None else running + batch_cost
        count += outputs.shape[0]
    cost = running / max(count, 1)
    cost = 0.5 * (cost + cost.T); cost.fill_diagonal_(0.0)
    nonzero = cost[cost > 1e-12]
    if nonzero.numel(): cost = cost / nonzero.median()
    return cost.detach().cpu()

def pairwise_factor_distance(factors):
    values = np.asarray(factors, dtype=float)
    return np.abs(values[:, None] - values[None, :])

def pairwise_root_chord_np(routes):
    routes = np.asarray(routes, dtype=float)
    roots = np.sqrt(np.clip(routes, 1e-12, None))
    roots /= np.linalg.norm(roots, axis=1, keepdims=True)
    delta = roots[:, None, :] - roots[None, :, :]
    return np.linalg.norm(delta, axis=-1) / math.sqrt(2.0)

```

```

def geometry_score_from_matrices(reference, observed):
    mask = np.triu(np.ones_like(reference, dtype=bool), k=1)
    rho = spearmanr(reference[mask], observed[mask]).statistic
    return float(np.nan_to_num(rho)), normalized_stress(reference, observed,
↪mask=mask)

def grouped_route_geometry(frame, cost=None):
    rows = []
    for source_id, group in frame.groupby("source_index"):
        group = group.sort_values("angle")
        factors = group.angle.to_numpy(float)
        routes = stack(group, "route")
        reference = pairwise_factor_distance(factors)
        if cost is None:
            observed = pairwise_root_chord_np(routes)
            metric = "nominal_root_chord"
        else:
            observed = pairwise_functional_route_distances(
                torch.tensor(routes, dtype=torch.float32),
                cost.float(),
                ↪
            )
            ↪epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),
            ↪
            ↪iterations=int(protocol["functional_route_metric"]["transport"]["iterations"]),
                ).numpy()
            metric = "functional_sinkhorn"
            rho, stress = geometry_score_from_matrices(reference, observed)
            rows.append({"source_index": source_id, "metric": metric, "rho": rho,
↪"stress": stress})
    return pd.DataFrame(rows)

def bootstrap_source_mean(values, seed):
    values = np.asarray(values, dtype=float)
    rng = np.random.default_rng(int(seed) % (2 ** 32))
    draws = np.asarray([
        values[rng.integers(0, len(values), len(values))].mean()
        for _ in range(SOURCE_BOOTSTRAP_SAMPLES)
    ])
    return float(values.mean()), float(np.quantile(draws, 0.025)), float(np.
↪quantile(draws, 0.975))

def local_continuity_rows(frame, common_cost, partition):

```

```

    rows=[]
    for source_id,group in frame.groupby("source_index"):
        group=group.sort_values("angle"); contexts=torch.
        ↪tensor(stack(group,"context"),dtype=torch.float32); routes=torch.
        ↪tensor(stack(group,"route"),dtype=torch.float32); angles=group.angle.
        ↪to_numpy(float)
        for i in range(len(group)-1):
            c0,c1=contexts[i:i+1],contexts[i+1:i+2]; r0,r1=routes[i:
        ↪i+1],routes[i+1:i+2]
            nominal=root_simplex_chord(r0,r1);
        ↪functional=entropic_transport_cost(r0,r1,common_cost.
        ↪float(),epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),iteration
        ↪dc=half_chord_distance(c0,c1)
            for metric,distance in
        ↪[("nominal",nominal),("functional",functional)]: rows.append({"source_index":
        ↪source_id,"partition":partition,"angle_a":angles[i],"angle_b":
        ↪angles[i+1],"metric":metric,"context_distance":float(dc),"route_distance":
        ↪float(distance),"continuity_ratio":
        ↪float(local_continuity_ratio(c0,c1,distance))})
        return pd.DataFrame(rows)

def continuity_summary(frame):
    rows=[]
    for keys,block in frame.
    ↪groupby(["model_seed","method","partition","metric"]):
        rows.append({"model_seed":keys[0],"method":keys[1],"partition":
        ↪keys[2],"metric":keys[3],"mean":block.continuity_ratio.mean(),"q90":block.
        ↪continuity_ratio.quantile(.90),"q95":block.continuity_ratio.quantile(.
        ↪.95),"q99":block.continuity_ratio.quantile(.99),"max":block.continuity_ratio.
        ↪max(),"pairs":len(block)})
    return pd.DataFrame(rows)

@torch.no_grad()
def perturbation_audit(model,method,model_seed,common_cost):
    rows=[]; model.eval()
    for x,_,_,source_ids in fixed_loader(15.
    ↪0,False,swap_eval_indices,False,batch_size=128):
        x=x.to(DEVICE); base=model(x,temperature=TAU)
        for scale in protocol["perturbations"]["context_noise_scales"]:

```



```

        g=torch.Generator(device=DEVICE); g.
        ↪manual_seed(model_seed*10000+int(float(scale)*1000)+91); noise=torch.
        ↪randn(base.context.shape,generator=g,device=DEVICE); noise-= (noise*base.
        ↪context).sum(-1,keepdim=True)*base.context; noise=F.normalize(noise,dim=-1);
        ↪changed=F.normalize(base.context+float(scale)*noise,dim=-1); route=model.
        ↪route_context(changed,temperature=TAU); nominal=root_simplex_chord(base.
        ↪route,route); functional=entropic_transport_cost(base.
        ↪route,route,common_cost.to(DEVICE),epsilon=.05,iterations=100);
        ↪z=route_weighted_synthesis(route,base.host_outputs); logits=model.
        ↪classifier(model.synthesis_norm(z))
        for i,sid in enumerate(source_ids): rows.append({"model_seed":
        ↪model_seed,"method":method,"source_index":int(sid),"scale":
        ↪float(scale),"nominal_change":float(nominal[i]),"functional_change":
        ↪float(functional[i]),"prediction_preserved":int(logits[i].argmax()==base.
        ↪logits[i].argmax())})
        return pd.DataFrame(rows)

@torch.no_grad()
def route_swap_audit(model,method,model_seed,common_cost):
    rows=[]; model.eval()
    for views,labels,factors,source_ids in
        ↪multi_loader(DENSE_EVAL_ANGLES,False,swap_eval_indices,False,batch_size=32):
        batch,n=views.shape[:2]; labels=labels.to(DEVICE); trace=model(views.
        ↪reshape(-1,*views.shape[2:]).to(DEVICE),temperature=TAU); routes=trace.route.
        ↪reshape(batch,n,NUM_HOSTS); hosts=trace.host_outputs.
        ↪reshape(batch,n,NUM_HOSTS,-1); logits=trace.logits.reshape(batch,n,-1);
        ↪base_logp=F.log_softmax(logits,-1); values=factors[0].cpu().numpy()
        for target in range(n):
            for donor in range(n):
                if target==donor: continue
                gap=abs(float(values[target]-values[donor])); category="near"
        ↪if gap<=30 else "far" if gap>=90 else "mid"
                if category=="mid": continue
                donor_route=routes[:,donor]; target_route=routes[:,target];
        ↪swapped=model.classifier(model.
        ↪synthesis_norm(route_weighted_synthesis(donor_route,hosts[:,target])));
        ↪swapped_logp=F.log_softmax(swapped,-1);
        ↪nominal=root_simplex_chord(target_route,donor_route);
        ↪functional=entropic_transport_cost(target_route,donor_route,common_cost.
        ↪to(DEVICE),epsilon=.05,iterations=100)

```

```

        for i,sid in enumerate(source_ids): rows.append({"model_seed":
↪model_seed,"method":method,"source_index":int(sid),"target_angle":
↪float(values[target]),"donor_angle":float(values[donor]),"gap":
↪gap,"category":category,"nominal_route_distance":
↪float(nominal[i]),"functional_route_distance":
↪float(functional[i]),"prediction_preserved":int(swapped[i].
↪argmax()==logits[i,target].argmax()),"true_log_prob_change":
↪float(swapped_logp[i,labels[i]]-base_logp[i,target,labels[i]]))}
    return pd.DataFrame(rows)

```

1.12 10. Low-capacity mediation probes and source-disjoint causal interventions

The v1.1.0 controls are retained, but all functional measurements use the common frozen cost matrix.

```

[12]: def fit_context_tangent(frame):
    X = stack(frame, "context").astype(float)
    y = frame.angle.to_numpy(float)
    X0 = X - X.mean(0); y0 = y - y.mean()
    ridge = 1e-6 * max(np.trace(X0.T @ X0) / max(X0.shape[1], 1), 1), 1.0)
    tangent = np.linalg.solve(X0.T @ X0 + ridge * np.eye(X0.shape[1]), X0.T @
↪y0)
    return tangent / max(np.linalg.norm(tangent), 1e-12)

def orthogonal_unit(tangent, seed):
    rng = np.random.default_rng(int(seed) % (2 ** 32))
    while True:
        vector = rng.normal(size=tangent.shape)
        vector -= np.dot(vector, tangent) * tangent
        norm = np.linalg.norm(vector)
        if norm > 1e-12: return vector / norm

def functional_route_error(actual, predicted, cost):
    return float(entropic_transport_cost(
        torch.tensor(actual, dtype=torch.float32),
        torch.tensor(predicted, dtype=torch.float32),
        cost.float(),
        ↪
↪epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),
        ↪
↪iterations=int(protocol["functional_route_metric"]["transport"]["iterations"]),
        ).mean())

```

```

def run_mediation_probe(method, model_seed, fit_frame, eval_frame, cost):
    X_train = stack(fit_frame, "context").astype(float)
    routes_train = stack(fit_frame, "route").astype(float)
    y_train = np.sqrt(np.clip(routes_train, 1e-12, None))
    X_eval = stack(eval_frame, "context").astype(float)
    routes_eval = stack(eval_frame, "route").astype(float)
    y_eval = np.sqrt(np.clip(routes_eval, 1e-12, None))
    tangent = fit_context_tangent(fit_frame)
    orth = orthogonal_unit(tangent, model_seed + 811)

    variants = {
        "full_context": (X_train, X_eval, y_train),
        "angle_shuffled": (X_train, X_eval, y_train[np.random.
↪default_rng(model_seed + 812).permutation(len(y_train))]),
        "tangent_removed": (
            X_train - np.outer(X_train @ tangent, tangent),
            X_eval - np.outer(X_eval @ tangent, tangent),
            y_train,
        ),
        "matched_orthogonal_component": (
            np.outer(X_train @ orth, orth),
            np.outer(X_eval @ orth, orth),
            y_train,
        ),
    }
    rows = []
    for variant, (train_x, eval_x, train_y) in variants.items():
        probe = Ridge(alpha=float(protocol["mediation_probe"]["ridge_alpha"])).
↪fit(train_x, train_y)
        prediction_root = probe.predict(eval_x)
        prediction_route = reconstruct_route_from_root_probe(prediction_root)
        rows.append({
            "model_seed": model_seed, "method": method, "variant": variant,
            "root_route_r2": float(r2_score(y_eval, prediction_root,
↪multioutput="variance_weighted")),
            "root_chord_error": float(pairwise_root_chord_np(np.
↪vstack([routes_eval, prediction_route]))[:len(routes_eval), len(routes_eval):
↪].diagonal().mean()),
            "functional_transport_error": functional_route_error(routes_eval,
↪prediction_route, cost),
            "evaluation_samples": len(eval_frame),
        })
    return pd.DataFrame(rows)

```

```

[13]: def orthogonal_directions(tangent, count, seed):
    rng = np.random.default_rng(int(seed) % (2 ** 32)); output = []
    while len(output) < count:

```

```

        vector = rng.normal(size=tangent.shape)
        vector -= np.dot(vector, tangent) * tangent
        norm = np.linalg.norm(vector)
        if norm > 1e-12: output.append(vector / norm)
    return output

def local_tangent(context, direction):
    projected = direction - (context * direction).sum(-1, keepdim=True) * \
    ↪ context
    return F.normalize(projected, dim=-1, eps=1e-8)

def root_route_jvp(router, context, direction):
    context = context.detach().requires_grad_(True)
    direction = direction.detach()
    def function(value):
        return torch.sqrt(router(value, temperature=TAU).clamp_min(1e-12))
    _, jvp = torch.autograd.functional.jvp(function, context, direction, \
    ↪ create_graph=False)
    return torch.linalg.vector_norm(jvp, dim=-1)

def source_bootstrap_ratio(frame, numerator, denominator, seed):
    rng = np.random.default_rng(int(seed) % (2 ** 32)); n = len(frame)
    values = []
    for _ in range(SOURCE_BOOTSTRAP_SAMPLES):
        sample = frame.iloc[rng.integers(0, n, n)]
        values.append(sample[numerator].mean() / max(sample[denominator].\
    ↪ mean(), 1e-12))
    point = frame[numerator].mean() / max(frame[denominator].mean(), 1e-12)
    return point, float(np.quantile(values, 0.025)), float(np.quantile(values, \
    ↪ 0.975))

def causal_mediation_probe(model, method, model_seed, fit_frame, cost):
    tangent_np = fit_context_tangent(fit_frame)
    orthogonal_np = orthogonal_directions(
        tangent_np, int(protocol["causal_protocol"]["orthogonal_controls"]), \
    ↪ model_seed + 901
    )
    raw_rows = []
    probe_angle = float(protocol["causal_protocol"]["probe_angle"])
    model.eval()
    for x, labels, _, source_ids in fixed_loader(probe_angle, False, \
    ↪ causal_eval_indices, False, batch_size=128):

```

```

x, labels = x.to(DEVICE), labels.to(DEVICE)
with torch.no_grad():
    base = model(x, temperature=TAU)
    base_logp = F.log_softmax(base.logits, -1).gather(1, labels[:,
↪None])).squeeze(1)
    base_pred = base.logits.argmax(1)
    tangent_global = torch.tensor(tangent_np, dtype=base.context.dtype,
↪device=DEVICE).expand_as(base.context)
    tangent = local_tangent(base.context, tangent_global)
    tangent_sensitivity = root_route_jvp(model.router, base.context,
↪tangent)
    orthogonal_locals = []
    orthogonal_sensitivities = []
    for direction in orthogonal_np:
        global_direction = torch.tensor(direction, dtype=base.context.
↪dtype, device=DEVICE).expand_as(base.context)
        local = local_tangent(base.context, global_direction)
        orthogonal_locals.append(local)
        orthogonal_sensitivities.append(root_route_jvp(model.router, base.
↪context, local))
    orthogonal_sensitivity = torch.stack(orthogonal_sensitivities).mean(0)

    for scale in protocol["causal_protocol"]["intervention_scales"]:
        scale = float(scale)
        with torch.no_grad():
            tangent_context = F.normalize(base.context + scale * tangent,
↪dim=-1)
            tangent_route = model.route_context(tangent_context,
↪temperature=TAU)
            nominal_t = root_simplex_chord(tangent_route, base.route)
            functional_t = entropic_transport_cost(
                tangent_route, base.route, cost.to(DEVICE),
                ↪
↪epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),
                ↪
↪iterations=int(protocol["functional_route_metric"]["transport"]["iterations"]),
            )
            z = torch.einsum("bh,bhd->bd", tangent_route, base.host_outputs)
            new_logits = model.classifier(model.synthesis_norm(z))
            new_logp = F.log_softmax(new_logits, -1).gather(1, labels[:,
↪None])).squeeze(1)
            new_pred = new_logits.argmax(1)

    nominal_controls = []; functional_controls = []
    for local in orthogonal_locals:

```

```

        control_context = F.normalize(base.context + scale * local,␣
↪dim=-1)

        control_route = model.route_context(control_context,␣
↪temperature=TAU)

        nominal_controls.append(root_simplex_chord(control_route,␣
↪base.route))

        functional_controls.append(entropic_transport_cost(
            control_route, base.route, cost.to(DEVICE),
            ␣
↪epsilon=float(protocol["functional_route_metric"]["transport"]["epsilon"]),
            ␣
↪iterations=int(protocol["functional_route_metric"]["transport"]["iterations"]),
        ))

        nominal_o = torch.stack(nominal_controls).mean(0)
        functional_o = torch.stack(functional_controls).mean(0)

        for index, source_id in enumerate(source_ids):
            raw_rows.append({
                "model_seed": model_seed, "method": method, "source_index":␣
↪int(source_id),
                "scale": scale,
                "nominal_tangent_change": float(nominal_t[index].cpu()),
                "nominal_orthogonal_change": float(nominal_o[index].cpu()),
                "functional_tangent_change": float(functional_t[index].
↪cpu()),
                "functional_orthogonal_change": float(functional_o[index].
↪cpu()),
                "tangent_jacobian_sensitivity":␣
↪float(tangent_sensitivity[index].detach().cpu()),
                "orthogonal_jacobian_sensitivity":␣
↪float(orthogonal_sensitivity[index].detach().cpu()),
                "prediction_preserved": int(base_pred[index] ==␣
↪new_pred[index]),
                "true_class_log_prob_change": float((new_logp[index] -␣
↪base_logp[index]).cpu()),
            })

        raw = pd.DataFrame(raw_rows)
        summary = []
        for scale, block in raw.groupby("scale"):
            nominal = source_bootstrap_ratio(block, "nominal_tangent_change",␣
↪"nominal_orthogonal_change", model_seed + int(abs(scale) * 1000) + 17)
            functional = source_bootstrap_ratio(block, "functional_tangent_change",␣
↪"functional_orthogonal_change", model_seed + int(abs(scale) * 1000) + 27)
            dmr = source_bootstrap_ratio(block, "tangent_jacobian_sensitivity",␣
↪"orthogonal_jacobian_sensitivity", model_seed + 37)

```

```

        preservation = bootstrap_source_mean(block.prediction_preserved.
↪to_numpy(float), model_seed + 47)
        summary.append({
            "model_seed": model_seed, "method": method, "scale": scale,
            "nominal_csr": nominal[0], "nominal_csr_ci_low": nominal[1],
↪"nominal_csr_ci_high": nominal[2],
            "functional_csr": functional[0], "functional_csr_ci_low":
↪functional[1], "functional_csr_ci_high": functional[2],
            "dmr": dmr[0], "dmr_ci_low": dmr[1], "dmr_ci_high": dmr[2],
            "prediction_preservation": preservation[0],
            "prediction_preservation_ci_low": preservation[1],
            "prediction_preservation_ci_high": preservation[2],
            "mean_true_class_log_prob_change": block.true_class_log_prob_change.
↪mean(),
        })
    return raw, pd.DataFrame(summary)

```

1.13 11. Allocation territories and fixed-host ablations

Because hosts are frozen, this stage measures allocation specificity rather than claiming that v1.1.1 created new host specialization.

```

[14]: @torch.no_grad()
def host_ecology_tables(model, method, model_seed, dense_trace, cost):
    routes = torch.tensor(stack(dense_trace, "route"), dtype=torch.float32)
    factors = torch.tensor(dense_trace.angle.to_numpy(float), dtype=torch.
↪float32)
    unique, territory = host_territory(routes, factors)
    specialization = host_specialization(territory)
    overlap = territory_overlap(territory)
    territory_rows = []
    for factor_index, angle in enumerate(unique.tolist()):
        for host in range(NUM_HOSTS):
            territory_rows.append({
                "model_seed": model_seed, "method": method, "angle": angle,
                "host": host, "mean_route_mass": float(territory[factor_index,
↪host]),
                "host_specialization": float(specialization[host]),
            })
    overlap_rows = [
        {"model_seed": model_seed, "method": method, "host_a": a, "host_b": b,
         "territory_overlap": float(overlap[a, b]), "functional_cost":
↪float(cost[a, b])}
        for a in range(NUM_HOSTS) for b in range(NUM_HOSTS)
    ]

    ablation_rows = []

```

```

rng = np.random.default_rng(model_seed + 1110)
for angle in DENSE_EVAL_ANGLES:
    angle_territory = territory[torch.argmax(torch.abs(unique - angle))]
    top_host = int(torch.argmax(angle_territory))
    random_host = int(rng.integers(0, NUM_HOSTS))
    total = 0; baseline_correct = 0
    per_host_correct = np.zeros(NUM_HOSTS, dtype=int)
    for x, y, _, _ in fixed_loader(angle, False, causal_eval_indices,
    ↪False, batch_size=128):
        x, y = x.to(DEVICE), y.to(DEVICE)
        trace = model(x, temperature=TAU)
        baseline_correct += int((trace.logits.argmax(1) == y).sum()); total
    ↪+= y.numel()
        for host in range(NUM_HOSTS):
            route = trace.route.clone(); route[:, host] = 0.0
            route = route / route.sum(-1, keepdim=True).clamp_min(1e-12)
            z = torch.einsum("bh,bhd->bd", route, trace.host_outputs)
            logits = model.classifier(model.synthesis_norm(z))
            per_host_correct[host] += int((logits.argmax(1) == y).sum())
        baseline_accuracy = baseline_correct / total
        for host in range(NUM_HOSTS):
            ablated_accuracy = per_host_correct[host] / total
            ablation_rows.append({
                "model_seed": model_seed, "method": method, "angle": angle,
    ↪"host": host,
                "baseline_accuracy": baseline_accuracy, "ablated_accuracy":
    ↪ablated_accuracy,
                "accuracy_drop": baseline_accuracy - ablated_accuracy,
                "is_top_host": int(host == top_host), "is_random_control":
    ↪int(host == random_host),
            })
        return pd.DataFrame(territory_rows), pd.DataFrame(overlap_rows), pd.
    ↪DataFrame(ablation_rows)

```

1.14 12. Per-seed execution

Each seed reproduces one aligned context, one common host bank, one common functional cost, and all R0-R5 treatments. SPD diagnostics are computed once from the shared context.

```

[15]: def source_geometry_summary(frame, method, model_seed, cost):
    rows = []
    for partition, angles in {"trained": TRAIN_ANGLES, "heldout_dense":
    ↪HELDOUT_ANGLES}.items():
        sub = frame[frame.angle.isin(angles)]
        for name, table in
    ↪[("nominal", grouped_route_geometry(sub, cost=None)), ("functional", grouped_route_geometry(sub
    ↪

```



```

        for metric in ["rho", "stress"]:
            ↵
            ↪mean, low, high = bootstrap_source_mean(table[metric], model_seed + (11 if ↵
            ↪metric == "rho" else 12)); rows.append({"model_seed": model_seed, "method":
            ↪method, "partition": partition, "geometry": name, "metric": metric, "mean":
            ↪mean, "source_ci_low": low, "source_ci_high": high, "source_blocks": len(table)})
            return pd.DataFrame(rows)

def run_seed(model_seed):
    print(f"\n===== MODEL SEED {model_seed} ====="); random.seed(model_seed); ↵
    ↪np.random.seed(model_seed); torch.manual_seed(model_seed)
    if torch.cuda.is_available(): torch.cuda.manual_seed_all(model_seed)
    ↵
    ↪source_model, checkpoint, context_hash, context_stage = train_aligned_context_checkpoint(model_s
    ↪context_bank = collect_context_bank(source_model, TRAIN_ANGLES, train_indices); ↵
    ↪bank_model, bank_state, bank_hash, bank_stage = train_common_host_bank(model_seed, checkpoint);
    ↪common_cost = build_common_host_cost(bank_model)
    models = {}; init_rows = []; stage_tables = []; eval_tables = []
    for method in METHODS:
        ↵
        ↪model, diag = build_routing_model(method, checkpoint, bank_state, context_bank, model_seed);
        ↪init_rows.append({"model_seed": model_seed, "method":
        ↪method, "router_parameters": parameter_count(model.router), **diag}); ↵
        ↪model, stage, evaluation = train_router_treatment(model_seed, method, model, context_hash, bank_has
        ↪models[method] = model; stage_tables.append(stage); eval_tables.
        ↪append(evaluation)
        stage = pd.concat(stage_tables, ignore_index=True); evaluation = pd.
        ↪concat(eval_tables, ignore_index=True)
        for _, block in stage.groupby("stage"): assert block.forward_images.
        ↪nunique() == 1 and block.optimizer_steps.nunique() == 1
        dense_trace = pd.
        ↪concat([collect_trace(models[m], m, model_seed, DENSE_EVAL_ANGLES, test_indices, include_hosts=T
        ↪for m in METHODS], ignore_index=True); ↵
        ↪preservation = preservation_table(dense_trace); assert preservation.
        ↪max_abs_context_delta.max() <= 1e-6 and preservation.max_abs_host_output_delta.
        ↪max() <= 1e-6
        spd_covariance, spd_pairs, spd_summary = spd_context_diagnostics(
            dense_trace[dense_trace.method == METHODS[0]], model_seed
        )
        probe_fit = pd.
        ↪concat([collect_trace(models[m], m, model_seed, TRAIN_ANGLES, probe_fit_indices) ↵
        ↪for m in METHODS], ignore_index=True); probe_eval = pd.
        ↪concat([collect_trace(models[m], m, model_seed, HELDOUT_ANGLES, probe_eval_indices) ↵
        ↪for m in METHODS], ignore_index=True); tangent_fit = pd.
        ↪concat([collect_trace(models[m], m, model_seed, TRAIN_ANGLES, tangent_fit_indices) ↵
        ↪for m in METHODS], ignore_index=True)

```

```

    geometry=[]; continuity=[]; probes=[]; causal_raw=[]; causal_summary=[];
    ↪perturb=[]; swaps=[]; territory=[]; overlap=[]; ablation=[];
    ↪source_tables={}; method_rows=[]; forgetting=[]; final_stage=int(evaluation.
    ↪stage.max())
    for method in METHODS:
        frame=dense_trace[dense_trace.method==method]
        geometry.
    ↪append(source_geometry_summary(frame,method,model_seed,common_cost))
        for partition,angles in {"trained":TRAIN_ANGLES,"heldout_dense":
    ↪HELDOUT_ANGLES}.items():
            sub=frame[frame.angle.isin(angles)];
    ↪source_tables[(method,partition,"nominal")]=grouped_route_geometry(sub,cost=None);
    ↪
    ↪source_tables[(method,partition,"functional")]=grouped_route_geometry(sub,cost=common_cost)
    ↪ c=local_continuity_rows(sub,common_cost,partition);
    ↪c["model_seed"]=model_seed; c["method"]=method; continuity.append(c)
            probes.append(run_mediation_probe(method,model_seed,probe_fit[probe_fit.
    ↪method==method],probe_eval[probe_eval.method==method],common_cost));
    ↪raw,summary=causal_mediation_probe(models[method],method,model_seed,tangent_fit[tangent_fit
    ↪method==method],common_cost); causal_raw.append(raw); causal_summary.
    ↪append(summary); perturb.
    ↪append(perturbation_audit(models[method],method,model_seed,common_cost));
    ↪swaps.append(route_swap_audit(models[method],method,model_seed,common_cost));
    ↪
    ↪t,o,a=host_ecology_tables(models[method],method,model_seed,frame,common_cost);
    ↪ territory.append(t); overlap.append(o); ablation.append(a)
            method_eval=evaluation[evaluation.method==method]
            for angle in TRAIN_ANGLES:
                seen_stage=CURRICULUM.index(angle); rows=method_eval[(method_eval.
    ↪angle==angle)&(method_eval.stage>=seen_stage)]; best=float(rows.accuracy.
    ↪max()); final=float(rows[rows.stage==final_stage].accuracy.iloc[0]);
    ↪forgetting.append({"model_seed":model_seed,"method":method,"angle":
    ↪angle,"best_accuracy":best,"final_accuracy":final,"forgetting":best-final})
                final=method_eval[method_eval.stage==final_stage]; method_rows.
    ↪append({"model_seed":model_seed,"method":method,"mean_heldout_accuracy":
    ↪final[final.angle.isin(HELDOUT_ANGLES)].accuracy.
    ↪mean(),"final_trained_accuracy":final[final.angle.isin(TRAIN_ANGLES)].
    ↪accuracy.mean(),"router_parameters":parameter_count(models[method].router)})
            paired=[]
            for contrast in PRIMARY_CONTRASTS:
                treatment,control=contrast["treatment"],contrast["control"]
                for partition in ["trained","heldout_dense"]:
                    for geometry_name in ["nominal","functional"]:
                        for metric in ["rho","stress"]:

```

```

        left=source_tables[(treatment,partition,geometry_name)][["source_index",metric]];
        right=source_tables[(control,partition,geometry_name)][["source_index",metric]];
        merged=left.merge(right,on="source_index",suffixes=("_t","_c"));
        delta=merged[f"{metric}_t"]-merged[f"{metric}_c"];
        mean,low,high=bootstrap_source_mean(delta,model_seed+211); paired.
        append({"model_seed":model_seed,"contrast":
        f"{treatment}_vs_{control}","partition":partition,"geometry":
        geometry_name,"metric":metric,"delta_mean":mean,"delta_ci_low":
        low,"delta_ci_high":high,"source_blocks":len(merged)})
        forgetting=pd.DataFrame(forgetting); method_summary=pd.
        DataFrame(method_rows).merge(forgetting.
        groupby(["model_seed","method"],as_index=False).forgetting.mean().
        rename(columns={"forgetting":
        "mean_forgetting"}),on=["model_seed","method"],how="left")
        return {"context_hash":context_hash,"bank_hash":bank_hash,"common_cost":
        common_cost.cpu(),"context_stage":context_stage,"bank_stage":
        bank_stage,"router_initialization":pd.DataFrame(init_rows),"stage":
        stage,"evaluation":evaluation,"trace":dense_trace,"preservation":
        preservation,"geometry":pd.
        concat(geometry,ignore_index=True),"paired_geometry":pd.
        DataFrame(paired),"continuity_raw":pd.
        concat(continuity,ignore_index=True),"mediation_probe":pd.
        concat(probes,ignore_index=True),"causal_raw":pd.
        concat(causal_raw,ignore_index=True),"causal_summary":pd.
        concat(causal_summary,ignore_index=True),"perturbation":pd.
        concat(perturb,ignore_index=True),"route_swap":pd.
        concat(swaps,ignore_index=True),"territory":pd.
        concat(territory,ignore_index=True),"overlap":pd.
        concat(overlap,ignore_index=True),"ablation":pd.
        concat(ablation,ignore_index=True),"forgetting":forgetting,"method_summary":
        method_summary,"spd_covariance":spd_covariance,"spd_pairs":
        spd_pairs,"spd_summary":spd_summary}

all_runs=[(seed,run_seed(seed)) for seed in MODEL_SEEDS]
print("Completed seeds:",[seed for seed,_ in all_runs])

```

\n===== MODEL SEED 0 =====

```

context {'model_seed': 0, 'stage': 0, 'current_angle': -60.0, 'seen_angles':
'[-60.0]', 'wall_seconds': 1.19204609999999705, 'forward_images': 1024,
'optimizer_steps': 16, 'loss': 1.8274404928088188, 'current_ce':
1.7848660051822662, 'anchor_ce': 0.0, 'route_geo': 0.0, 'context_geo': 0.0,
'context_spread': 0.002500000176951289, 'fiber': 0.0, 'centroid': 0.0,
'host_diversity': 0.8489894159138203}
context {'model_seed': 0, 'stage': 1, 'current_angle': -30.0, 'seen_angles':
'[-60.0, -30.0]', 'wall_seconds': 2.4518257939998875, 'forward_images': 2048,

```

```

'optimizer_steps': 16, 'loss': 1.556731328368187, 'current_ce':
1.3552666492760181, 'anchor_ce': 1.2216982170939445, 'route_geo':
0.12866627983748913, 'context_geo': 0.04250857466831803, 'context_spread':
0.00020963217411917867, 'fiber': 0.5014275535941124, 'centroid':
0.01165978214999086, 'host_diversity': 0.7302504368126392}
context {'model_seed': 0, 'stage': 2, 'current_angle': 0.0, 'seen_angles':
'[-60.0, -30.0, 0.0]', 'wall_seconds': 2.8606778229996053, 'forward_images':
3072, 'optimizer_steps': 16, 'loss': 0.7175729237496853, 'current_ce':
0.5171247823163867, 'anchor_ce': 1.152430597692728, 'route_geo':
0.2539558978751302, 'context_geo': 0.050096603110432625, 'context_spread':
3.274957009580248e-05, 'fiber': 0.47606404684484005, 'centroid':
0.036922108905855566, 'host_diversity': 0.5829776097089052}
context {'model_seed': 0, 'stage': 3, 'current_angle': 60.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0]', 'wall_seconds': 3.1486032209995756,
'forward_images': 4096, 'optimizer_steps': 16, 'loss': 2.0344742611050606,
'current_ce': 1.8286719769239426, 'anchor_ce': 1.183510396629572, 'route_geo':
0.3343885689973831, 'context_geo': 0.07600971916690469, 'context_spread':
5.0942230700457e-07, 'fiber': 0.429350771009922, 'centroid':
0.09437775751575828, 'host_diversity': 0.4044990334659815}
context {'model_seed': 0, 'stage': 4, 'current_angle': 30.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0, 30.0]', 'wall_seconds': 3.818547421000403,
'forward_images': 5120, 'optimizer_steps': 16, 'loss': 0.8970313258469105,
'current_ce': 0.6945051997900009, 'anchor_ce': 1.2496512308716774, 'route_geo':
0.2352123148739338, 'context_geo': 0.07707270327955484, 'context_spread': 0.0,
'fiber': 0.4996451511979103, 'centroid': 0.07309799338690937, 'host_diversity':
0.3539069704711437}
\n===== MODEL SEED 1 =====
context {'model_seed': 1, 'stage': 0, 'current_angle': -60.0, 'seen_angles':
'[-60.0]', 'wall_seconds': 1.0163274139995337, 'forward_images': 1024,
'optimizer_steps': 16, 'loss': 1.9558866396546364, 'current_ce':
1.9131375774741173, 'anchor_ce': 0.0, 'route_geo': 0.0, 'context_geo': 0.0,
'context_spread': 0.002500000176951289, 'fiber': 0.0, 'centroid': 0.0,
'host_diversity': 0.8524815179407597}
context {'model_seed': 1, 'stage': 1, 'current_angle': -30.0, 'seen_angles':
'[-60.0, -30.0]', 'wall_seconds': 1.6066206809991854, 'forward_images': 2048,
'optimizer_steps': 16, 'loss': 1.6175028681755066, 'current_ce':
1.412396915256977, 'anchor_ce': 1.2397688403725624, 'route_geo':
0.12748053157702088, 'context_geo': 0.039695704937912524, 'context_spread':
0.0002491801969881635, 'fiber': 0.5564186070114374, 'centroid':
0.013868008594727144, 'host_diversity': 0.7176944762468338}
context {'model_seed': 1, 'stage': 2, 'current_angle': 0.0, 'seen_angles':
'[-60.0, -30.0, 0.0]', 'wall_seconds': 2.1746777019998262, 'forward_images':
3072, 'optimizer_steps': 16, 'loss': 0.7259656488895416, 'current_ce':
0.5248025851324201, 'anchor_ce': 1.1940825879573822, 'route_geo':
0.23109073750674725, 'context_geo': 0.047820870066061616, 'context_spread':
3.415597138256565e-05, 'fiber': 0.47823939844965935, 'centroid':
0.02468682902690489, 'host_diversity': 0.574312187731266}
context {'model_seed': 1, 'stage': 3, 'current_angle': 60.0, 'seen_angles':

```

```

'[-60.0, -30.0, 0.0, 60.0]', 'wall_seconds': 3.194836115999351,
'forward_images': 4096, 'optimizer_steps': 16, 'loss': 2.1036443263292313,
'current_ce': 1.887081801891327, 'anchor_ce': 1.2245997302234173, 'route_geo':
0.3739459365606308, 'context_geo': 0.09927346231415868, 'context_spread':
2.444048291749823e-06, 'fiber': 0.4764918442815542, 'centroid':
0.06965764000779018, 'host_diversity': 0.38946177437901497}
context {'model_seed': 1, 'stage': 4, 'current_angle': 30.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0, 30.0]', 'wall_seconds': 4.866632979999849,
'forward_images': 5120, 'optimizer_steps': 16, 'loss': 0.9296926483511925,
'current_ce': 0.7216502167284489, 'anchor_ce': 1.2836312800645828, 'route_geo':
0.2755517475306988, 'context_geo': 0.07046858500689268, 'context_spread': 0.0,
'fiber': 0.532947288826108, 'centroid': 0.019987852196209133, 'host_diversity':
0.34861025772988796}
\n===== MODEL SEED 2 =====
context {'model_seed': 2, 'stage': 0, 'current_angle': -60.0, 'seen_angles':
'[-60.0]', 'wall_seconds': 0.9422984449993237, 'forward_images': 1024,
'optimizer_steps': 16, 'loss': 1.947095662355423, 'current_ce':
1.9044951051473618, 'anchor_ce': 0.0, 'route_geo': 0.0, 'context_geo': 0.0,
'context_spread': 0.002500000176951289, 'fiber': 0.0, 'centroid': 0.0,
'host_diversity': 0.8495110683143139}
context {'model_seed': 2, 'stage': 1, 'current_angle': -30.0, 'seen_angles':
'[-60.0, -30.0]', 'wall_seconds': 1.7619473159993504, 'forward_images': 2048,
'optimizer_steps': 16, 'loss': 1.5397205278277397, 'current_ce':
1.3412109091877937, 'anchor_ce': 1.2136708721518517, 'route_geo':
0.1188340550288558, 'context_geo': 0.029194870847277343, 'context_spread':
0.00015080858565852395, 'fiber': 0.5210066586732864, 'centroid':
0.012672376593400259, 'host_diversity': 0.7129624262452126}
context {'model_seed': 2, 'stage': 2, 'current_angle': 0.0, 'seen_angles':
'[-60.0, -30.0, 0.0]', 'wall_seconds': 3.2930096840009355, 'forward_images':
3072, 'optimizer_steps': 16, 'loss': 0.700000949203968, 'current_ce':
0.5038357488811016, 'anchor_ce': 1.1804726421833038, 'route_geo':
0.20282090362161398, 'context_geo': 0.04489377071149647, 'context_spread':
1.4217901815882783e-05, 'fiber': 0.4760640971362591, 'centroid':
0.019051062859944068, 'host_diversity': 0.5717995557934046}
context {'model_seed': 2, 'stage': 3, 'current_angle': 60.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0]', 'wall_seconds': 3.3558021369990456,
'forward_images': 4096, 'optimizer_steps': 16, 'loss': 2.05204014480114,
'current_ce': 1.8358505442738533, 'anchor_ce': 1.1888744793832302, 'route_geo':
0.36272923089563847, 'context_geo': 0.1138483565300703, 'context_spread':
7.844577498104799e-07, 'fiber': 0.4719220530241728, 'centroid':
0.1457324679940939, 'host_diversity': 0.3752317223697901}
context {'model_seed': 2, 'stage': 4, 'current_angle': 30.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0, 30.0]', 'wall_seconds': 4.015360526000222,
'forward_images': 5120, 'optimizer_steps': 16, 'loss': 0.902299165725708,
'current_ce': 0.6928039211779833, 'anchor_ce': 1.2419192790985107, 'route_geo':
0.2993121109902859, 'context_geo': 0.07140571461059153, 'context_spread': 0.0,
'fiber': 0.5321044810116291, 'centroid': 0.06148658273741603, 'host_diversity':
0.3710397854447365}

```

```

\n===== MODEL SEED 3 =====
context {'model_seed': 3, 'stage': 0, 'current_angle': -60.0, 'seen_angles':
'[-60.0]', 'wall_seconds': 1.1059982709994074, 'forward_images': 1024,
'optimizer_steps': 16, 'loss': 1.93125419318676, 'current_ce':
1.8885361105203629, 'anchor_ce': 0.0, 'route_geo': 0.0, 'context_geo': 0.0,
'context_spread': 0.002500000176951289, 'fiber': 0.0, 'centroid': 0.0,
'host_diversity': 0.8518618531525135}
context {'model_seed': 3, 'stage': 1, 'current_angle': -30.0, 'seen_angles':
'[-60.0, -30.0]', 'wall_seconds': 1.5921619740001915, 'forward_images': 2048,
'optimizer_steps': 16, 'loss': 1.5041144490242004, 'current_ce':
1.303204108029604, 'anchor_ce': 1.222976803779602, 'route_geo':
0.11352329840883613, 'context_geo': 0.03561043168883771, 'context_spread':
0.00016137384523062792, 'fiber': 0.5570544730871916, 'centroid':
0.015584335917992576, 'host_diversity': 0.7011856399476528}
context {'model_seed': 3, 'stage': 2, 'current_angle': 0.0, 'seen_angles':
'[-60.0, -30.0, 0.0]', 'wall_seconds': 2.2192918860000646, 'forward_images':
3072, 'optimizer_steps': 16, 'loss': 0.6588918585330248, 'current_ce':
0.4661181913688779, 'anchor_ce': 1.182922426611185, 'route_geo':
0.1819728333503008, 'context_geo': 0.041285997023805976, 'context_spread':
1.0304579129982727e-05, 'fiber': 0.5085904225707054, 'centroid':
0.015085913975781295, 'host_diversity': 0.5194240286946297}
context {'model_seed': 3, 'stage': 3, 'current_angle': 60.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0]', 'wall_seconds': 3.27528960500058, 'forward_images':
4096, 'optimizer_steps': 16, 'loss': 1.9793449118733406, 'current_ce':
1.7711356207728386, 'anchor_ce': 1.2790776640176773, 'route_geo':
0.25543148536235094, 'context_geo': 0.09215586120262742, 'context_spread':
6.457858034991659e-07, 'fiber': 0.529900262132287, 'centroid':
0.09696160990279168, 'host_diversity': 0.2839940916746855}
context {'model_seed': 3, 'stage': 4, 'current_angle': 30.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0, 30.0]', 'wall_seconds': 5.085346083000331,
'forward_images': 5120, 'optimizer_steps': 16, 'loss': 0.9016415923833847,
'current_ce': 0.7023260463029146, 'anchor_ce': 1.2517721429467201, 'route_geo':
0.22530884761363268, 'context_geo': 0.07110582338646054, 'context_spread': 0.0,
'fiber': 0.5556021053344011, 'centroid': 0.07502247369848192, 'host_diversity':
0.2593129314482212}
\n===== MODEL SEED 4 =====
context {'model_seed': 4, 'stage': 0, 'current_angle': -60.0, 'seen_angles':
'[-60.0]', 'wall_seconds': 1.0009473280006205, 'forward_images': 1024,
'optimizer_steps': 16, 'loss': 1.8844543397426605, 'current_ce':
1.841995544731617, 'anchor_ce': 0.0, 'route_geo': 0.0, 'context_geo': 0.0,
'context_spread': 0.002500000176951289, 'fiber': 0.0, 'centroid': 0.0,
'host_diversity': 0.8466754965484142}
context {'model_seed': 4, 'stage': 1, 'current_angle': -30.0, 'seen_angles':
'[-60.0, -30.0]', 'wall_seconds': 1.620572595998965, 'forward_images': 2048,
'optimizer_steps': 16, 'loss': 1.5733205527067184, 'current_ce':
1.3762545697391033, 'anchor_ce': 1.1878099963068962, 'route_geo':
0.12663606088608503, 'context_geo': 0.04095319740008563, 'context_spread':
0.000171660228716064, 'fiber': 0.5041259415447712, 'centroid':

```

```

0.005447497329811313, 'host_diversity': 0.7207750156521797}
context {'model_seed': 4, 'stage': 2, 'current_angle': 0.0, 'seen_angles':
'[-60.0, -30.0, 0.0]', 'wall_seconds': 2.147421370998927, 'forward_images':
3072, 'optimizer_steps': 16, 'loss': 0.7092668786644936, 'current_ce':
0.511904938146472, 'anchor_ce': 1.1592403873801231, 'route_geo':
0.2413955256342888, 'context_geo': 0.04861066001467407, 'context_spread':
2.4153145290028988e-05, 'fiber': 0.45447569712996483, 'centroid':
0.020683884766185656, 'host_diversity': 0.5735614523291588}
context {'model_seed': 4, 'stage': 3, 'current_angle': 60.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0]', 'wall_seconds': 4.281218201998854,
'forward_images': 4096, 'optimizer_steps': 16, 'loss': 2.1761028692126274,
'current_ce': 1.9411439970135689, 'anchor_ce': 1.2091854326426983, 'route_geo':
0.43152554146945477, 'context_geo': 0.16377543797716498, 'context_spread':
9.244485186077345e-06, 'fiber': 0.4695909135043621, 'centroid':
0.2319379122927785, 'host_diversity': 0.3886672016233206}
context {'model_seed': 4, 'stage': 4, 'current_angle': 30.0, 'seen_angles':
'[-60.0, -30.0, 0.0, 60.0, 30.0]', 'wall_seconds': 4.5100681929998245,
'forward_images': 5120, 'optimizer_steps': 16, 'loss': 0.9243446514010429,
'current_ce': 0.7014490962028503, 'anchor_ce': 1.3020000085234642, 'route_geo':
0.31394029036164284, 'context_geo': 0.09928690921515226, 'context_spread':
5.3136723323632395e-06, 'fiber': 0.5687614642083645, 'centroid':
0.08600807655602694, 'host_diversity': 0.3726819735020399}
Completed seeds: [0, 1, 2, 3, 4]

```

1.15 13. Seed aggregation, repair gates, and falsification

The primary router treatment is R5. R1 is the smooth low-capacity reference; R0 is the operational reference; R3 is the sharp-router historical control.

[16]:

```

def concat_key(key): return pd.concat([run[key] for _, run in
    ↪all_runs], ignore_index=True)
context_stage_df=concat_key("context_stage");
    ↪bank_stage_df=concat_key("bank_stage");
    ↪router_initialization_df=concat_key("router_initialization");
    ↪stage_df=concat_key("stage"); evaluation_df=concat_key("evaluation");
    ↪trace_df=concat_key("trace"); preservation_df=concat_key("preservation");
    ↪geometry_df=concat_key("geometry");
    ↪paired_geometry_df=concat_key("paired_geometry");
    ↪continuity_raw_df=concat_key("continuity_raw");
    ↪mediation_probe_df=concat_key("mediation_probe");
    ↪causal_raw_df=concat_key("causal_raw");
    ↪causal_summary_df=concat_key("causal_summary");
    ↪perturbation_df=concat_key("perturbation");
    ↪route_swap_df=concat_key("route_swap"); territory_df=concat_key("territory");
    ↪overlap_df=concat_key("overlap"); ablation_df=concat_key("ablation");
    ↪forgetting_df=concat_key("forgetting");
    ↪per_seed_method_df=concat_key("method_summary");
    ↪continuity_summary_df=continuity_summary(continuity_raw_df);
    ↪spd_covariance_df=concat_key("spd_covariance");
    ↪spd_pairs_df=concat_key("spd_pairs");
    ↪spd_summary_df=concat_key("spd_summary")
compute_summary_df=stage_df.groupby(["model_seed", "method"], as_index=False).
    ↪agg(total_forward_images=("forward_images", "sum"), total_optimizer_steps=("optimizer_steps",
def seed_ci(values, confidence=.95):
    values=np.asarray(values, dtype=float); n=len(values); mean=float(values.
    ↪mean()); sd=float(values.std(ddof=1)) if n>1 else 0.0
    if n<2: return mean, mean, mean, sd, n
    half=float(stats.t.ppf((1+confidence)/2, df=n-1)*sd/math.sqrt(n)); return
    ↪mean, mean-half, mean+half, sd, n
aggregate_effect_rows=[]
for keys, block in paired_geometry_df.
    ↪groupby(["contrast", "partition", "geometry", "metric"]):
    mean, low, high, sd, n=seed_ci(block.delta_mean); favorable=block.delta_mean>0
    ↪if keys[3]=="rho" else block.delta_mean<0; aggregate_effect_rows.
    ↪append({"contrast":keys[0], "partition":keys[1], "geometry":keys[2], "metric":
    ↪keys[3], "mean_seed_effect":mean, "seed_ci_low":low, "seed_ci_high":
    ↪high, "seed_sd":sd, "seed_count":n, "favorable_seed_fraction":float(np.
    ↪mean(favorable))})
aggregate_effect_df=pd.DataFrame(aggregate_effect_rows)
aggregate_method_rows=[]
for method, block in per_seed_method_df.groupby("method"):
    for metric in
    ↪["mean_heldout_accuracy", "final_trained_accuracy", "mean_forgetting"]:

```



```

        mean,low,high,sd,n=seed_ci(block[metric]); aggregate_method_rows.
        ↳append({"method":method,"metric":metric,"mean":mean,"seed_ci_low":
        ↳low,"seed_ci_high":high,"seed_sd":sd,"seed_count":n})
    aggregate_method_df=pd.DataFrame(aggregate_method_rows)
    causal_seed_df=causal_summary_df[causal_summary_df.scale!=0].
        ↳groupby(["model_seed","method"],as_index=False).
        ↳agg(mean_functional_csr=("functional_csr","mean"),min_functional_csr_ci_low=("functional_cs
    swap_summary_df=route_swap_df.
        ↳groupby(["model_seed","method","category"],as_index=False).
        ↳agg(mean_functional_distance=("functional_route_distance","mean"),mean_log_prob_change=("tr

    spd_pivot = spd_summary_df.pivot(index="model_seed", columns="metric",
        ↳values="stress")
    spd_delta = spd_pivot["centroid_plus_log_euclidean"] - spd_pivot["centroid"]
    spd_mean, spd_low, spd_high, spd_sd, spd_n = seed_ci(spd_delta)
    spd_aggregate_df = pd.DataFrame([
        "contrast": "centroid_plus_log_euclidean_vs_centroid",
        "metric": "stress",
        "mean_seed_effect": spd_mean,
        "seed_ci_low": spd_low,
        "seed_ci_high": spd_high,
        "seed_sd": spd_sd,
        "seed_count": spd_n,
        "favorable_seed_fraction": float(np.mean(spd_delta < 0)),
    ])

```

```

[17]: def get_effect(contrast, partition, geometry, metric):
    rows = aggregate_effect_df[
        (aggregate_effect_df.contrast == contrast)
        & (aggregate_effect_df.partition == partition)
        & (aggregate_effect_df.geometry == geometry)
        & (aggregate_effect_df.metric == metric)
    ]
    if rows.empty:
        raise KeyError((contrast, partition, geometry, metric))
    return rows.iloc[0]

primary = "r5_smooth_residual_continuity_vs_r1_linear"
functional_rho = get_effect(primary, "heldout_dense", "functional", "rho")
functional_stress = get_effect(primary, "heldout_dense", "functional", "stress")

continuity = continuity_summary_df[
    (continuity_summary_df.partition == "heldout_dense")
    & (continuity_summary_df.metric == "functional")
].pivot(index="model_seed", columns="method", values="q95")

```

```

continuity_delta = continuity["r5_smooth_residual_continuity"] -
    ↪continuity["r1_linear"]
continuity_mean, continuity_low, continuity_high, _, _ =
    ↪seed_ci(continuity_delta)

swap = swap_summary_df.pivot_table(
    index=["model_seed", "method"], columns="category",
    ↪values="mean_functional_distance"
).reset_index()
r5_swap = swap[swap.method == "r5_smooth_residual_continuity"]
swap_fraction = float(np.mean(r5_swap.far > r5_swap.near))

r5_causal = causal_seed_df[causal_seed_df.method ==
    ↪"r5_smooth_residual_continuity"]
r5 = per_seed_method_df[per_seed_method_df.method ==
    ↪"r5_smooth_residual_continuity"].set_index("model_seed")
r0 = per_seed_method_df[per_seed_method_df.method == "r0_mlp"].
    ↪set_index("model_seed")
margin = float(protocol["candidate_gates"]["operational_equivalence_margin"])
acc_delta = r5.mean_heldout_accuracy - r0.mean_heldout_accuracy
forgetting_delta = r5.mean_forgetting - r0.mean_forgetting
q95_tolerance = float(protocol["local_continuity"]["q95_tolerance"])

candidate_gates = pd.DataFrame([
    {"gate": "C0_context_and_common_bank_frozen", "passed":
    ↪bool(preservation_df.max_abs_context_delta.max() <= 1e-6 and preservation_df.
    ↪max_abs_host_output_delta.max() <= 1e-6), "status": "protocol_integrity"},
    {"gate": "C0_equal_router_compute", "passed": bool(all(block.
    ↪total_forward_images.nunique() == 1 and block.total_optimizer_steps.
    ↪nunique() == 1 for _, block in compute_summary_df.groupby("model_seed"))),
    ↪"status": "protocol_integrity"},
    {"gate": "C1_heldout_functional_stress_reduction", "passed":
    ↪bool(functional_stress.seed_ci_high < 0 and functional_stress.
    ↪favorable_seed_fraction >= 0.80), "status": "candidate_only"},
    {"gate": "C1_heldout_functional_order", "passed": bool(functional_rho.
    ↪seed_ci_low > 0 and functional_rho.favorable_seed_fraction >= 0.80),
    ↪"status": "candidate_only"},
    {"gate": "C1_local_continuity_non_degradation", "passed":
    ↪bool(continuity_high <= q95_tolerance), "status": "candidate_only"},
    {"gate": "C2_route_swap_order", "passed": bool(swap_fraction >= 0.80),
    ↪"status": "candidate_only"},
    {"gate": "C4_functional_causal_specificity", "passed": bool((r5_causal.
    ↪min_functional_csr_ci_low > 1).mean() >= 0.80 and r5_causal.
    ↪mean_functional_csr.mean() > 1.5), "status": "candidate_only"},

```

```

    {"gate": "C4_identity_preservation", "passed": bool((r5_causal.
↪min_prediction_preservation >= 0.95).mean() >= 0.80), "status":␣
↪"candidate_only"},
    {"gate": "C6_operational_non_degradation", "passed": bool((np.
↪abs(acc_delta) <= margin).mean() >= 0.80 and (forgetting_delta <= margin).
↪mean() >= 0.80), "status": "equivalence_not_superiority"},
    {"gate": "GEO_SPD_covariance_adds_value", "passed": bool(spd_high < 0 and␣
↪float((spd_delta < 0).mean()) >= 0.80), "status":␣
↪"diagnostic_candidate_only"},
    {"gate": "C3_mature_host_specialization", "passed": False, "status":␣
↪"not_tested_hosts_frozen"},
    {"gate": "C5_memory_transport", "passed": False, "status":␣
↪"not_implemented"},
    {"gate": "operational_superiority", "passed": False, "status":␣
↪"not_preregistered_as_primary"},
])
candidate_gates

```

```

[17]:
      gate  passed \
0      C0_context_and_common_bank_frozen    True
1              C0_equal_router_compute    True
2      C1_heldout_functional_stress_reduction    False
3              C1_heldout_functional_order    False
4      C1_local_continuity_non_degradation    False
5              C2_route_swap_order    True
6      C4_functional_causal_specificity    False
7              C4_identity_preservation    True
8      C6_operational_non_degradation    False
9      GEO_SPD_covariance_adds_value    True
10      C3_mature_host_specialization    False
11      C5_memory_transport    False
12      operational_superiority    False

      status
0      protocol_integrity
1      protocol_integrity
2      candidate_only
3      candidate_only
4      candidate_only
5      candidate_only
6      candidate_only
7      candidate_only
8      equivalence_not_superiority
9      diagnostic_candidate_only
10     not_tested_hosts_frozen
11     not_implemented

```

12 not_preregistered_as_primary

1.16 14. Export immutable evidence and verification handoff

The export includes router evidence, continuity diagnostics, SPD covariance matrices, pairwise SPD distances, and manifest files.

```
[18]: try:
    git_sha = subprocess.check_output(["git", "rev-parse", "HEAD"], text=True).
    ↪strip()
except Exception:
    git_sha = "unknown"
root_out = Path("results/v1_1_2")
root_out.mkdir(parents=True, exist_ok=True)

for seed, run in all_runs:
    seed_out = root_out / f"seed_{seed}"
    seed_out.mkdir(parents=True, exist_ok=True)
    for filename, key in {
        "context_pretraining_stage_logs.csv": "context_stage",
        "common_bank_stage_logs.csv": "bank_stage",
        "router_initialization.csv": "router_initialization",
        "routing_stage_logs.csv": "stage",
        "staged_evaluation.csv": "evaluation",
        "dense_trace.pkl": "trace",
        "preservation.csv": "preservation",
        "route_geometry.csv": "geometry",
        "paired_route_geometry_effects.csv": "paired_geometry",
        "local_continuity_raw.csv": "continuity_raw",
        "mediation_probe_controls.csv": "mediation_probe",
        "causal_mediation_raw.csv": "causal_raw",
        "causal_mediation_summary.csv": "causal_summary",
        "perturbation_sensitivity.csv": "perturbation",
        "route_swap_effects.csv": "route_swap",
        "allocation_territory.csv": "territory",
        "host_overlap_and_cost.csv": "overlap",
        "host_ablation.csv": "ablation",
        "forgetting.csv": "forgetting",
        "method_summary.csv": "method_summary",
        "spd_context_covariances.csv": "spd_covariance",
        "spd_pairwise_context_distances.csv": "spd_pairs",
        "spd_context_geometry.csv": "spd_summary",
    }.items():
        value = run[key]
        value.to_pickle(seed_out / filename) if filename.endswith(".pkl") else ↪
        ↪value.to_csv(seed_out / filename, index=False)
        torch.save(run["common_cost"], seed_out / "common_host_cost_matrix.pt")
```

```

(seed_out / "checkpoint_hashes.json").write_text(json.dumps({
    "context_checkpoint_hash": run["context_hash"],
    "common_host_bank_hash": run["bank_hash"],
}, indent=2), encoding="utf-8")

for filename, frame in {
    "context_pretraining_stage_logs.csv": context_stage_df,
    "common_bank_stage_logs.csv": bank_stage_df,
    "router_initialization.csv": router_initialization_df,
    "routing_stage_logs.csv": stage_df,
    "staged_evaluation.csv": evaluation_df,
    "preservation.csv": preservation_df,
    "route_geometry.csv": geometry_df,
    "paired_route_geometry_effects.csv": paired_geometry_df,
    "aggregate_seed_effects.csv": aggregate_effect_df,
    "local_continuity_raw.csv": continuity_raw_df,
    "local_continuity_summary.csv": continuity_summary_df,
    "mediation_probe_controls.csv": mediation_probe_df,
    "causal_mediation_summary.csv": causal_summary_df,
    "causal_seed_summary.csv": causal_seed_df,
    "perturbation_sensitivity.csv": perturbation_df,
    "route_swap_effects.csv": route_swap_df,
    "route_swap_summary.csv": swap_summary_df,
    "allocation_territory.csv": territory_df,
    "host_overlap_and_cost.csv": overlap_df,
    "host_ablation.csv": ablation_df,
    "forgetting.csv": forgetting_df,
    "per_seed_method_summary.csv": per_seed_method_df,
    "aggregate_method_summary.csv": aggregate_method_df,
    "compute_summary.csv": compute_summary_df,
    "spd_context_covariances.csv": spd_covariance_df,
    "spd_pairwise_context_distances.csv": spd_pairs_df,
    "spd_context_geometry.csv": spd_summary_df,
    "spd_aggregate_effects.csv": spd_aggregate_df,
    "gate_summary.csv": candidate_gates,
    "sensory_history.csv": pd.DataFrame(sensory_history),
}.items():
    frame.to_csv(root_out / filename, index=False)

claim_manifest = {
    "schema_version": "1.1",
    "version": NOTEBOOK_VERSION,
    "execution_status": "pilot_pending",
    "status": "Complete C0 implementation; five-seed v1.1.2 pilot pending_
↪execution.",
    "primary_treatment": "r5_smooth_residual_continuity",
    "eligible_if_gates_pass": [

```

```

        "held-out functional stress reduction",
        "held-out functional order",
        "local continuity non-degradation",
        "route-swap ordering",
        "functional causal specificity",
        "operational non-degradation",
        "SPD covariance diagnostic value",
    ],
    "non_claims": protocol["non_claims"],
}
(root_out / "claim_manifest.json").write_text(json.dumps(claim_manifest,
    ↪indent=2), encoding="utf-8")

manifest = {
    "notebook_version": NOTEBOOK_VERSION,
    "build_revision": BUILD_REVISION,
    "package_version": geometry_mmalls.__version__,
    "git_sha": git_sha,
    "run_profile": RUN_PROFILE,
    "model_seeds": MODEL_SEEDS,
    "context_checkpoint_hashes": {str(seed): run["context_hash"] for seed, run
    ↪in all_runs},
    "common_bank_hashes": {str(seed): run["bank_hash"] for seed, run in
    ↪all_runs},
    "split_manifest": split_manifest,
    "sensory_accuracy": sensory_accuracy,
    "methods": METHOD_SPECS,
    "primary_contrasts": PRIMARY_CONTRASTS,
    "common_functional_metric": True,
    "hosts_frozen_across_router_treatments": True,
    "continuity_uses_external_factor_labels": False,
    "spd_axis": "diagnostic_candidate_only",
    "source_v111_results": "results/v1_1_1",
    "verification_profile": "verification/profiles/geometry_g1_v1_1_2.yaml",
    "warning": "Qualification claims require executed evidence and independent
    ↪verification.",
    "timestamp": time.time(),
}
(root_out / "run_manifest.json").write_text(json.dumps(manifest, indent=2),
    ↪encoding="utf-8")
(root_out / "environment.txt").write_text("\n".join([
    f"python={sys.version}", f"platform={platform.platform()}",
    f"torch={torch.__version__}", f"device={DEVICE}",
]), encoding="utf-8")
print("Exported v1.1.2 evidence to", root_out.resolve())

```

Exported v1.1.2 evidence to /content/geometry-mmalls-g1/results/v1_1_2

1.17 15. Interpretation discipline and roadmap

A successful v1.1.2 result would show that bounded residual routing can retain functional structure without recreating the sharp local tails of R2/R3. A positive SPD diagnostic would show only that context covariance adds measurable factor information beyond centroid geometry.

It would not yet establish mature host specialization, memory transport, multi-objective Energy-Guided control, RL control, a learned Riemannian manifold, or quantum advantage.

- **v1.1.3:** route-function memory trajectories and transport;
- **v1.1.4:** selective causal host adaptation and recovery;
- **G2.1:** explicit bottom-up host feedback;
- **TPUT:** goal-conditioned forward-backward control.

```
[19]: import shutil
from pathlib import Path
import datetime
import os

# Define the source directory to be zipped
source_dir = REPO_DIR # This is '/content/geometry-mmalls-g1'

# Get the notebook filename from the 'notebook_path' variable
# This assumes 'notebook_path' was set in a previous cell, which it was.
# However, it might be None if the previous cell failed to find the path.

# Robustly get the current notebook path
notebook_id = os.environ.get('COLAB_JUPYTER_ITEM_ID')
if notebook_id:
    # Assuming notebook title is available in a variable named 'notebook_title'
    ↪or similar
    # For this specific notebook, we can infer it from the text cells.
    notebook_title = "Geometry_MMALS_G1_BridgeIsolation_v1_1_1" # Corrected
    ↪notebook title
    notebook_path = REPO_DIR / 'notebooks' / f'{notebook_title}.ipynb' # Added
    ↪'notebooks' subdirectory
else:
    # Fallback for other environments or if ID is not available
    # Search for the .ipynb file in /content/ or known locations
    notebook_path_candidates = [
        REPO_DIR / 'notebooks' / 'Geometry_MMALS_G1_BridgeIsolation_v1_1_1.
        ↪ipynb',
        # Add other potential paths if known
    ]
    notebook_path = next((p for p in notebook_path_candidates if p.exists()),
        ↪None)

if notebook_path is None:
    # If still not found, raise an error or handle appropriately
```